

19. Bölüm

Kurumsal Yazılımlar

19.1 Kurumsal Yazılım İçin Gereklikler

§19.1.1 İş Analizi ve Gereksinim Analizi

İş analizi (business analysis), kurumun hedeflerine ulaşması için gerekli olan değişimleri yönetme disiplinidir. Sadece bir yazılım projesiyle sınırlı değildir. Büyük resmi inceler. Organizasyonel yapı değişikliği, yeni bir pazara giriş veya maliyet azaltma stratejileri gibi konular da iş analizinin konusudur. "Şu an neredeyiz?", "Nereye gitmek istiyoruz?" ve "Varmak istediğimiz yer için en mantıklı yatırım nedir?" gibi soruların cevabını arar. Bunun için; Pazar analizi, rakip analizi, yatırım getirisi hesaplamaları ve paydaş yönetimi konuları içerir.

Gereksinim analizi (requirement analysis), iş analizinin bir alt kümesidir. İş analizi tarafından belirlenen "ihtiyaç", somut bir çözüme dönüştürülmesi için gereken tüm teknik ve fonksiyonel detayların ayıklandığı süreçtir. "İş hedefine ulaşmak için sistemin veya ürünün tam olarak ne yapması gerekiyor?" sorusunun cevabını verir. Bunun için; Kullanıcı senaryoları yazma, veri modelleri oluşturma, kısıtları belirleme ve onay süreçlerini yönetme gibi konuları irdeler.

Bir bankanın müşteri kaybını azaltmak istediğini varsayalım:

- ◊ **İş Analizi Kısmı:** Analist, verileri inceleyerek müşterilerin mobil uygulamanın yavaşlığı nedeniyle rakip bankaya geçtiğini tespit eder. Çözüm olarak mobil uygulamanın yenilenmesi gerektiğini önerir ve bunun bankaya ne kadar kazandıracağını hesaplar.
- ◊ **Gereksinim Analizi Kısmı:** Analist, yeni uygulamada "hızlı transfer" butonunun nerede olacağını, transferin kaç saniyede tamamlanması gerektiğini, hata mesajlarının nasıl görüneceğini ve hangi güvenlik protokollerinin (örneğin yüz tanıma, parmak izi veya kimlik kartı gibi) kullanılacağını detaylandırır.
- ◊ **Özetle:** İş analizi rotayı çizer ve varış noktasını (hedefi) belirler; gereksinim analizi ise o hedefe ulaşmak için aracın nasıl çalışması gerektiğini tarif eden kullanım kılavuzunu yazar.

İş analizi sürecinde **gereksinim analizi**, paydaşlardan toplanan ham bilgilerin (ihtiyaçlar, istekler, kısıtlar) işlenerek; tutarlı, eksiksiz ve doğrulanmış birer "gereksinim" haline getirildiği kritik aşamadır. Süreci bir mutfağa benzetirsek; paydaşlardan gelen bilgiler "ham malzemeler", gereksinim analizi ise bu malzemelerin ayıklanıp, doğranıp pişirilerek sunuma hazır hale getirilmesidir.

Özellik	İş Analizi (Business Analysis)	Gereksinim Analizi (Requirements Analysis)
Kapsam	Çok geniştir; kurumun stratejisini ve tüm iş süreçlerini kapsar.	Daha dardır; belirli bir çözüm veya proje özelindeki detaylara odaklanır.
Odak Noktası	İş problemlerini tanımlamak, fırsatları belirlemek ve değer yaratmak.	Toplanan bilgileri yapılandırmak, detaylandırmak ve teknik dile hazırlamak.
Zamanlama	Proje başlamadan önce başlar, proje boyunca ve sonrasında devam eder.	Genellikle çözümün kapsamı netleştikten sonra (planlama aşamasında) başlar.
Çıktı	İş vakası (business case), Fizibilite raporu, Stratejik plan.	Gereksinim dokümanı (BRD/PRD), Kullanıcı hikayeleri, Akış diyagramları.

Tablo 19.1: İş Analizi-Gereksinim Analizi Karşılaştırması

§19.1.2 Gereksinim Analizi Adımları

Bu süreç doğrusal değil, döngüsel bir yapıdadır. Temel adımlar şunlardır:

1. **Modelleme ve Düzenleme:** Toplanan bilgiler kategorize edilir. Bu aşamada görsel modeller kullanılarak karmaşık süreçler basitleştirilir.
 - ◊ Süreç Modelleri: İş akış diyagramları (**flowcharts**).
 - ◊ Veri Modelleri: Varlık-İlişki Diyagramları (ERD).
 - ◊ Kullanım Senaryoları: Use Case diyagramları.

2. **Gereksinimlerin Önceliklendirilmesi:** Tüm gereksinimler aynı anda yapılamaz. Kaynaklar ve zaman kısıtlıdır. Yaygın olarak **MoSCoW** tekniği kullanılır:
 - ◊ **Must have:** Olmazsa olmazlar.
 - ◊ **Should have:** Olsa iyi olur (Yüksek öncelik).
 - ◊ **Could have:** Olsa da olur (Düşük öncelik).
 - ◊ **Won't have:** Bu aşamada yapılmayacaklar.
3. **Doğrulama ve Onay (verification & validation):** Gereksinimlerin kalitesi kontrol edilir:
 - ◊ **Doğrulama:** Gereksinimler standartlara uygun mu? (Net, ölçülebilir, test edilebilir mi?)
 - ◊ **Geçerlilik:** Bu gereksinim gerçekten iş ihtiyacını karşılıyor mu?

	Tür	Açıklama ve Örnek
	İş Gereksinimleri	Kurumun üst düzey hedefleri: "Yıllık operasyonel maliyeti %10 azaltmak."
	Paydaş Gereksinimleri	Belirli bir kullanıcı grubunun ihtiyacı: "Depo sorumlusu stokları anlık görebilmeli."
	Çözüm (Fonksiyonel)	Sistemin ne yapması gerektiği: "Sistem, stok 10'un altına düştüğünde uyarı vermeli."
	Çözüm (Fonksiyonel Olmayan)	Sistemin nasıl çalışması gerektiği: "Sistem 2 saniyenin altında yanıt vermelidir."

Tablo 19.2: Gereksinim Türleri

§19.1.3 İyi Bir Gereksinimin Özellikleri

Analiz sonunda ortaya çıkan her bir gereksinim şu kriterleri karşılamalıdır:

- ◊ **Atomik:** Daha küçük parçalara bölünemez olmalı.
- ◊ **Net (unambiguous):** Herkes tarafından aynı şekilde anlaşılmalı.
- ◊ **Test Edilebilir:** Başarılı olup olmadığı ölçülebilmeli.
- ◊ **Tutarlı:** Diğer gereksinimlerle çelişmemeli.
- ◊ **İzlenebilir:** Hangi iş ihtiyacından doğduğu takip edilebilmeli.

Bunun için kullanılan teknikler

1. **Kullanıcı Hikayeleri (user stories):** "Bir [rol] olarak, [amaç] istiyorum, böylece [fayda] sağlayacağım" kalıbı.
2. **Kabul Kriterleri (acceptance criteria):** Bir özelliğin "tamam" sayılması için gereken şartlar.
3. **Prototipleme:** Gereksinimleri somutlaştırmak için ekran taslakları hazırlama.

Gereksinim analizi sadece "yazmak" değildir; paydaşlar arasındaki çatışmaları yönetmek ve "gerçekten neye ihtiyaç duyulduğunu" bulup çıkarmaktır.

İş analizi sonrasında çözüm bir yazılım geliştirme olacaksa buna **kurumsal yazılım (enterprise software)** adı verilir. Bu durumda **gereksinim analizi (requirement analysis)**, geliştirilecek yazılım için başvuru kitabıdır ve **yazılımın kabul ve test noktasında en önemli rolü oynar.**

§19.1.4 Gereksinim Analizi ve Modelleme Yol Haritası

Bir **kurumsal yazılım** sisteminin inşası için kurumda iş analizi yapılır. İş analizinde aşağıdaki modeller sırasıyla çıkarılır;

- ◊ **İşlevsel model (functional model)** çıkarılır. Kullanıcılara ne (**what**) yapmak istedikleri sorulur ve **use-case** diyagramları oluşturulur.
- ◊ **Yapısal model (structural model)** çıkarılır. İşlemlerin kimler (**who**) tarafından yapılacağı belirlenir ve bu kapsamda sınıf diyagramları oluşturulur.
- ◊ **Davranış modeli (behavioral model)** oluşturulur. İşlemlerin kim tarafından nasıl (**how**) ve hangi sırada yapılacağı belirlenir ve silsile diyagramları oluşturulur.
- ◊ **Mimari modelleme (architectural model)** oluşturulur. Bu modelde nesnelerin nasıl ve nerede (**where**) bir araya geleceği üzerine durulur ve yayılma ve paket diyagramları oluşturulur. Bu modelde iki gereklilik detaylı incelenir;
 - Birincisi iş süreçlerini ya da kullanıcı isteklerini yerine getiren işlevsel gerekliliklerdir (**functional requirements**).
 - İkincisi ise yazılımla birlikte ortaya çıkan performans, güvenlik (**security**), günlük (**logging**) ve

benzeri gerekliliklerdir (**non-functional requirements**).

Ancak güncel çevik süreçlerde yukarıda sırasıyla yapılması gerektiği anlatılan modeller **çoğu zaman eş zamanlı ve artımlı (incremental)** ilerler.

§19.1.5 Kurumsal Uygulama Geliştirmenin Temel Bileşenleri

Kurumların karmaşık iş süreçlerini dijital ortama taşıyan ve geniş ölçekli operasyonları yöneten sistemler kurumsal uygulamalar (**enterprise application**) olarak adlandırılır. Başarılı bir kurumsal yazılım projesi inşa edebilmek için üç temel unsura ihtiyaç duyulur:

- ◊ **Takım (team)**: Yazılımın fikir aşamasından yayına alınmasına kadar görev yapan; iş analisti, sistem mimarı, yazılım geliştirici, test uzmanı ve proje yöneticisi gibi farklı uzmanlıklara sahip yetkin bir gruptur.
- ◊ **Yazılım Geliştirme Yaşam Döngüsü SDLC (Software Development Life Cycle)**: Projenin yönetim stratejisini belirleyen metodolojilerdir. İhtiyaca göre doğrusal ilerleyen Şelale Modeli (**waterfall model**), yinelemeli bir yapı sunan RUP (**Rational Unified Process**) veya günümüzde en yaygın tercih edilen esnek **Çevik (agile)** modellerden biri seçilir. Bu modeller, karmaşıklığı yönetmek ve riskleri azaltmak için bir yol haritası sunar.
- ◊ **Teknoloji (technology)**: Kurumsal yazılımların ihtiyaç duyduğu güvenlik, veri tabanı yönetimi, ağ iletişimi ve işlem yönetimi gibi standart altyapı hizmetlerini hazır olarak sunan platformlardır. Bu teknolojiler (frameworks), geliştiriciye karmaşık altyapı detaylarıyla uğraşmak yerine doğrudan iş mantığına (**business logic**) odaklanma imkânı tanır. Günümüzde bu alanda en çok tercih edilen ekosistemler Jakarta EE (**Enterprise Edition**) ve .NET (**.NET Framework/Core**) platformlarıdır.

Yazılım geliştirme sırasında bu üç şeyi çok iyi seçmemiz halinde başarılı bir kurumsal yazılım ortaya çıkar. Burada özellikle teknoloji üzerinde durmak gerekir. Jakarta EE ve .NET gibi iki büyük teknolojinin programcıya sunduğu bileşenler arkasında birçok sunucu çalışır. Bunların bazıları aşağıda verilmiştir;

- ◊ **Kurumsal yazılım geliştirmede kullanılan modüler bileşenleri sunan sunucular**: Jakarta Enterprise Beans, COM (**Component Object Model**), COM+ gibi teknolojileri sağlayan .NET Bileşenleri
- ◊ **Kullanıcı, bilgisayar, yazıcı ve benzeri öğeleri içeren kurumsal dizin sunucuları**: JNDI (**Java Naming and Directory Interface**), Microsoft Active Directory
- ◊ **Kurumsal Uygulamada bileşenlerin birbirine ileti (message) gönderip almalarını sağlayan servisler**: Jakarta Messaging API, MSMQ (**Microsoft Message Queue**)
- ◊ **Web Sunucuları üzerinde bileşenlerin çalışmasını sağlayan servisler**: JSP (**Jakarta Server Pages**), ASP.NET Core (.NET platformu için Web Uygulama Çatısı)
- ◊ **Mevcut kurumsal uygulamaları birbirine bağlayan servisler**: Jakarta Connectors, JDBC (**Java Database Connectivity**), Java XML, Jakarta Transactions, ADO.NET (**Active Data Objects for .NET**), WCF (**Windows Communication Foundation**)

Kurumlarda JEE ile .NET teknolojileri arasında seçim yapmak için, kurumun sahip olduğu donanım altyapısı ve yukarıda belirtilen servisleri sağlayan yazılım kaynakları irdelenmelidir.

Tüm projelerde olduğu gibi kurumlarda geliştirilen uygulamalar da proje olarak ele alınır ve yönetilirler. Proje yönetimi, belirli bir hedefe ulaşmak için zaman, bütçe, kaynaklar ve kalite kısıtları altında yürütülen işleri planlama, yürütme, izleme ve tamamlama sürecidir. Kısaca doğru işi, doğru şekilde, doğru zamanda ve doğru kaynaklarla yapmak demektir. Proje yönetimi genellikle beş ana aşamada yürütülür:

1. **Başlatma (initiation)**: Projenin amacı, kapsamı ve paydaşları belirlenir. Proje yöneticisi atanır ve gerekirse fizibilite çalışması yapılır. Proje yöneticisi, proje planı hazırlamak ve güncel tutar, takımı yönetir, iletişimi sağlar, riskleri öngörür ve önlem alır, paydaşlarla iletişimi sürdürür ve teslimatları zamanında gerçekleştirir.
2. **Planlama (planning)**: Proje hedefleri, teslimatlar belirlenir. **Zaman çizelgesi (gantt chart)** ve alt kırılımlar oluşturulur. Kaynak planlaması, risk yönetimi, bütçe, iletişim planı hazırlanır.
3. **Yürütme (execution)**: Takım kurulur, görevler dağıtılır. İşler gerçekleştirilir, yazılım geliştirme, inşa gibi ana faaliyetler yapılır. İletişim ve koordinasyon sağlanır.
4. **İzleme ve Kontrol (monitoring and controlling)**: Proje ilerlemesi takip edilir. Gecikmeler, sapmalar analiz edilir. Riskler yönetilir, gerekirse planlar revize edilir.
5. **Kapanış (closing)**: Proje sonuçlandırılır. Teslimatlar yapılır. Geri bildirim alınır, raporlama yapılır, dersler çıkarılır.

Proje yönetimini yapmak için **Microsoft Project, Jira, Trello, Asana, ClickUp, Notion, Monday.com, Smartsheet ve Wrike** gibi araçlar kullanılır.

Bir yazılım projesini yönetmek için yukarıdaki yönetim aşamalarını içerecek şekilde yazılım geliştirme yaşam döngüsü SDLC (**Software Development Life Cycle**) ile yapılır. Yazılım geliştirme modeli olarak da

adlandırılan SDLC, kaliteli yazılım tasarlamak, geliştirmek ve test etmek için kullanılan yapılandırılmış bir süreçtir. SDLC, bir yazılım mühendisi veya geliştiricisi tarafından çeşitli aşamalarda gerçekleştirilecek görevleri belirtir. Son ürünün müşterinin beklentilerini karşılayabilmesini ve tanımlanan bütçeye uymasını sağlar¹.

Burada **yazılım mühendisliği (software engineering)** devreye girer. Yazılım mühendisliği, yazılım sistemlerinin planlanması, tasarımı, geliştirilmesi, test edilmesi, dağıtımı ve bakımı ile ilgilenir. Yazılım mühendisi;

- ◊ Yazılım geliştirme veya kısaca programlama yapar,
- ◊ Yazılım mimarisi belirler,
- ◊ Yazılım testini yapar ve kalitesini belirler,
- ◊ Yazılım proje yönetimini yapar,
- ◊ Yazılım yaşam döngüsü sürecini (SDLC) yürütür,
- ◊ Geliştirme ve işletim **DevOps (Development and Operations)** süreci ile sürekli entegrasyon/sürekli dağıtım **CI/CD (Continuous Integration/Continuous Delivery)** süreçlerini yürütür.

19.2 Yazılım Geliştirme Modelleri

Günümüzde birçok yazılım geliştirme modeli yani SDLC bulunmaktadır. Bu kitapta bunlardan ilki olan şelale modeli ile günümüzde çok kullanılan çevik modele değinilecektir.

§19.2.1 Şelale Modeli

Şelale modeli (waterfall model) 1970 yılında *Winston W. Royce* tarafından tanımlanmıştır. Bu modelde bir sonraki aşamaya geçmek için, bir önceki aşamanın tamamlanması gerekir. Bir sonraki aşamaya geçmeden, içinde bulunulan aşamanın tüm dokümanları hazırlanır. Bu model, **klasik yaşam döngüsü (classic life cycle)**, doğru sıralı (**linear sequential**) model olarak da adlandırılır. Şelale modelinin aşamaları;

1. **Gereksinim Analizi (requirement analysis)**: Müşteri ihtiyaçları toplanır ve belgelenir. Projenin kapsamı belirlenir, Projenin hedef ve amaçları belirlenir. Projenin kaynakları planlanır. Fonksiyonel ve teknik gereksinimler belirlenir. Gereksinimler gözden geçirilir ve kabul koşulları belirlenir.
2. **Sistem ve Yazılım Tasarımı (design)**: Yazılım mimarisi ve sistem yapısı planlanır. Veri tabanı, ara yüz, iş mantığı gibi bileşenler tasarlanır. Düşük ve yüksek düzey tasarım yapılır. Düşük Seviye Tasarım LLD (**Low Level Design**), yazılım geliştirme sürecinde detaylı sistem bileşenlerinin ve bunların etkileşimlerinin belirtildiği ve bunların uygulandığı bir aşamadır. Yüksek Seviye Tasarım HLD (**High Level Design**), bir sistemin genel yapısının planlandığı uygulamaların geliştirilmesinde ilk adım olup, esas olarak sistemin farklı bileşenlerinin, kodlama ve uygulama hakkında bilgi sahibi olmadan birlikte nasıl çalıştığına odaklanır. Bu, projede yer alan herkesin hedefleri anlamasına yardımcı olur ve geliştirme sırasında iyi iletişim sağlar.
3. **Uygulama ya da Kodlama (implementation)**: Tasarlanan yapıya uygun olarak yazılım geliştirme yapılır. Kod standardı belirlenir yani programlama dilleri burada belirlenir. Kod ölçeklendirme yapılır. Kod ölçeklendirme, uygulamanın daha fazla kullanıcı, daha fazla veri veya daha yoğun trafik ile etkili şekilde çalışabilmesi için yapılan düzenlemelerdir. Sürüm kontrolü belirlenir. Kodun gözden geçirilmesi için süreç belirlenir.
4. **Test Etme (testing)**: Kodlanan yazılım test edilir. Genel sistem testi yanında manuel test ve otomatik testler yapılabilir. Varsa hatalar düzeltilir, kalite kontrolü sağlanır.
5. **Dağıtım (deployment)**: Yazılımın uygun sürümü son kullanıcıya sunulur. Kurulumlar ve eğitimler yapılabilir.
6. **Bakım (maintenance)**: Kullanım sırasında oluşan hatalar düzeltilir. Geri bildirimler sonrası güncellemeler ve iyileştirmeler yapılır.

Bu modelde yazılıma ilişkin dokümantasyon çok net ve kapsamlıdır. Bir aşama tamamlanmadan sıradakine geçilmez. Süreç kontrollüdür. Gereksinimi değişmeyen büyük projelerde kullanılır. Yönetimi ve izlenebilirliği kolaydır. Ancak geri dönmek zordur. Bir hata sonradan fark edilirse süreç baştan sarar. Değişen gereksinimler varsa uygun değildir. Bu nedenle esnek değildir. Teslimat çok geç ortaya çıkar yani müşteri ürünü en sonunda görür.

§19.2.2 Çevik Model

Çevik model (agile model), şelale modelinin aksine temel olarak proje süresince gelen değişen isteklere hızla uyum sağlamak için tasarlanmıştır. Günümüzde çok tercih edilen modellerden biridir. Bu model, yazılım geliştirme süreçlerinde esnekliği, müşteri memnuniyetini ve hızlı teslimatı ön planda tutan tekrarlamalı (**iterative**) bir yaklaşımdır. Büyük bir projeyi küçük parçalara ayırarak, adım adım geliştirme ve sürekli geri bildirimle iyileştirme sağlar. Bu model, değişen müşteri ihtiyaçlarına hızlıca adapte olabilmek için tasarlanmış, sürekli iletişim, erken teslimat ve kademeli gelişim ilkelerine dayanan bir geliştirme modelidir.

¹<https://www.geeksforgeeks.org/software-development-life-cycle-sdlc/>

Çevik modelde projeyi yönetmek için proje personelinin bazı değerleri (**values**) benimsemesi gerekir;

1. Bireyler ve etkileşimler, süreçler ve araçlardan daha önemlidir.
2. Çalışan yazılım, kapsamlı dokümantasyondan daha önemlidir. Burada, dokümantasyonun "yok sayılması" değil, "kodun kendisinin bir doküman gibi (**self-documenting code**) yazılması çok önemlidir.
3. Müşteri ile iş birliği, sözleşme pazarlığından daha önemlidir.
4. Değişime karşılık vermek, bir planı izlemekten daha önemlidir.

Bu değerler ile **12 adet ilkeye uyulur**. Bunlar **çevik manifestoda** (**agile manifesto**) yer alır;

1. Müşteri memnuniyeti, erken ve sürekli yazılım teslimatıyla sağlanır. Müşteriye sık aralıklarla çalışan yazılım sunarak geri bildirim alınır ve güven kazanılır.
2. Değişen gereksinimler, projenin geç aşamalarında bile kabul edilir. Çevik süreçler, değişime yanıt vererek müşterinin rekabet avantajını korumasına yardımcı olur.
3. Çalışan yazılım, birkaç haftada bir, mümkün olan en kısa sürede teslim edilir. **Kısa döngülerle** (**sprint**) geliştirme yapılır.
4. İş birliği, iş birimlerinden (müşteriden) ve geliştiricilerden oluşan bir ekip arasında sürekli olmalıdır. Müşteri ve geliştirici ekip sürekli iletişim halinde olur.
5. Motivasyonu yüksek bireyler etrafında projeler inşa edilir. Ekip üyelerine güvenilir, destekleyici bir ortam sunulur ve onlara güvenilir.
6. Yüz yüze iletişim, bilgi aktarımının en etkili yoludur. Özellikle aynı fiziksel ortamda çalışan ekiplerde hızlı karar alınmasını sağlar.
7. Çalışan yazılım, ilerlemenin birincil ölçüsüdür. Raporlar ya da belgeler değil, gerçek çalışan yazılım önemlidir.
8. Çevik süreçler sürdürülebilir geliştirmeyi teşvik eder. Ekip, sabit bir hızda uzun süre çalışabilmelidir. Aşırı mesai değil, dengeli tempo hedeflenir.
9. Teknik mükemmellik ve iyi tasarım, çevikliği artırır. Kaliteli kod ve temiz mimari, değişimlere uyumu kolaylaştırır.
10. Basitlik esastır. Yapılmaması gereken işleri yapmaktan kaçınmak. Gereksiz özellik ve aşırı mühendislikten uzak durulur.
11. En iyi mimariler, gereksinimler ve tasarımlar, kendi kendini organize eden ekiplerden çıkar. Ekip içindeki bireyler, en uygun çözümleri birlikte bulur.
12. Ekip düzenli aralıklarla nasıl daha verimli olabileceğini düşünür ve davranışlarını buna göre uyarlar. **Sürekli iyileştirme** (**retrospective**) ile gelişim sağlanır.

Çevik proje yönetiminde ön tanımlı roller vardır;

1. Ürün Sahibi (**Product Owner**): Müşteri ihtiyaçlarını belirler. Ürünün ne olacağına karar verir, iş önceliklerini belirler.
2. Şef (**Scrum Master**): Takımın çevik kurallarına uymasını sağlar. Engelleri kaldırır, kolaylaştırıcı rolündedir.
3. Geliştirme Takımı (**Development Team**): Yazılımı analiz eder tasarlar, kodlar, test eder.

Çevik modelde proje, **sprint** adı verilen kısa geliştirme döngülerine ayrılır. Her sprint sonunda çalışan bir yazılım parçası müşteriye teslim edilir. Bunun için;

- ◇ İş listesi (**Product Backlog**) hazırlanır.
- ◇ Her bir kısa döngü için toplantı (**Sprint Planning**) yapılır.
- ◇ Yazılım takımı, 1 ila 4 haftalık kısa döngü süresince belirlenen işleri yapar.
- ◇ Günlük toplantılar (**Daily Scrum**) yani günlük toplantılar ile ilerleme izlenir.
- ◇ Kısa döngü gözden geçirme toplantısı (**Sprint Review**) ile müşteriye sunum yapılır.
- ◇ Sürekli iyileştirme (**Sprint Retrospective**) ile ekip kendi sürecini değerlendirir ve iyileştirir.

Bir kurumda çevik proje yönetimi yapabilmek için kurumun buna hazırlanması gerekir;

- ◇ **Kurum Kültürü**: Çevik proje yönetimi, hiyerarşik ve katı yönetim anlayışı olan kurumlarda zor olur. Bu yüzden üst yönetimin çevik felsefesini benimsemesi gerekir. Takımlara güven, şeffaflık ve sürekli iyileştirme desteklenmelidir. "Komuta-kontrol" yerine "hizmetkar liderlik" yaklaşımı benimsenmelidir.
- ◇ **Kurum İçi Eğitim**: Tüm ilgili ekipler (yazılım geliştiriciler, testçiler, analistler, ürün sahipleri, yöneticiler) çevik eğitimi almalıdır. **Scrum**, **Kanban**, **Lean** gibi çerçeveler tanıtılmalı ve hangi modelin kuruma uygun olduğuna karar verilmelidir.
- ◇ **Roller**: **Scrum** kullanılıyorsa roller atanmalıdır; **Product Owner**, **Scrum Master**, **Development Team**.

- ◇ **Takım:** Küçük, çapraz fonksiyonel ve çok disiplinli (**multi-disipliner**) takımlar oluşturulmalıdır. Her takım kendi içinde analiz, tasarım, geliştirme ve test yapabilir durumda olmalıdır.
- ◇ **Sprint Yönetimi:** Scrum uygulanıyorsa;
 - **Kısa Döngü Planlama** (Sprint Planning): İşlerin belirlenmesi ve sprint hedefinin tanımlanması.
 - **Günlük Toplantı** (Daily Stand-up): Kısa günlük toplantılarla durum değerlendirmesi.
 - **Kısa Döngü Gözden Geçirmesi** (Sprint Review): Ürün sahipleri ve paydaşlarla yapılan toplantı (**demo**).
 - **Sürekli İyileştirme Toplantısı** (Sprint Retrospective): Ekip içi iyileştirme toplantısı.
- ◇ **Agile süreçleri destekleyen yazılımlar:** **Jira, Trello, Azure DevOps, ClickUp** ve **Asana** gibi araçlarla iş takibi yapılabilir. Ayrıca CI/CD altyapıları kurulmalıdır.
- ◇ **Pilot Proje Deneyimi:** Öncelikle küçük bir projede çevik yöntem uygulanır. Deneyim kazanılır, süreçler test edilir. Başarılar ve hatalar analiz edilerek diğer ekiplerde yaygınlaştırılır.
- ◇ **Sürekli İyileştirme (Kaizen):** Ekipler düzenli olarak süreçlerini gözden geçirip iyileştirme önerileri üretmelidir. Geri bildirim döngüleri (müşteriden ve ekip içinden) önemlidir.
- ◇ **Başarı ve Performans Ölçütleri:** Çevik ekiplerin performansı birey bazında değil, takım bazında ölçülür. Teslim süreleri, müşteri memnuniyeti, iş değeri odaklı metrikler kullanılır.

- ◇ Çevik modelde **müşteri memnuniyeti yüksektir.**
- ◇ Hızlı teslimat ve **erken geri bildirim** alınır.
- ◇ **Değişime açık** ve uyumlu yazılım geliştirilir.
- ◇ Takım içi **iletişim güçlüdür.**
- ◇ Kalite artar, çünkü sürekli test edilir, ancak **sürekli müşteri katılımı gerekir.**
- ◇ **Belirsiz projelerde zaman ve maliyet tahmini zordur.**
- ◇ **Deneyimsiz ekiplerde karmaşıklık ortaya çıkar!**
- ◇ Dokümantasyon önemsiz gibi görülüp ihmal edilebilir. Bu proje **sonrası bakım maliyetini ciddi oranda artırır!**

19.3 Modern Kurumsal Yazılımların Özellikleri

Günümüzde geliştirilen modern uygulamaların üç temel özelliği vardır:

1. **Katmanlı mimari (layered architecture):** İstemci/sunucu ya da n-katmanlı modeldir. Katmanlı yapı, sorumlulukları birbirinden ayırarak bağımlılığı azaltır; ancak katman sayısı arttıkça sistemin yapısal **karmaşıklığı (complexity)** yönetilebilirlik açısından dikkatle planlanmalıdır. Sanılanın aksine, katman sayısı arttıkça sistemin toplam karmaşıklığı teknik olarak artar; ancak bu yapı, bileşenler arasındaki bağımlılığı (**coupling**) azaltarak yönetilebilirliği kolaylaştırır. Her katmanın belirli bir görevi olması, bakım ve hata ayıklama süreçlerini hızlandırır.
2. **Bileşen kullanımı (component using):** Yazılımın modüler parçalardan oluşması, kodun farklı projelerde **tekrar kullanılabilirliğini (reusing)** artırır ve bakım maliyetlerini düşürür.
3. **Dağıtık programlama (distributed programming):** Bu yaklaşım sayesinde sistemin **ölçeklenebilirliği (scalability)** artar. Yazılım, iş yükünü birden fazla bilgisayar veya sunucu üzerine çeşitli görevlerle dağıtarak yüksek erişilebilirlik sağlar. Bu yaklaşım, sistemin ölçeklenebilirliğini (**scalability**) ve hata toleransını (**fault tolerance**) maksimize eder. Günümüzde bu özellik kendini en net **mikroservis mimarileri** ve **bulut bilişim (cloud computing)** teknolojilerinde gösterir. Bir sunucuda sorun yaşandığında, yük otomatik olarak diğerlerine dağıtılarak sistemin kesintisiz çalışması sağlanır.

Dağıtık programlamanın günümüzdeki en popüler ve olgun yansıması olan **mikroservis mimarisi (microservices architecture)**, büyük ve yekpare (**monolithic**) bir uygulamanın, her biri kendi başına çalışan, belirli bir iş mantığını yürüten ve birbirleriyle hafif protokoller (genellikle HTTP/REST veya mesaj kuyrukları) üzerinden haberleşen küçük servisler bütününe bölünmesidir.

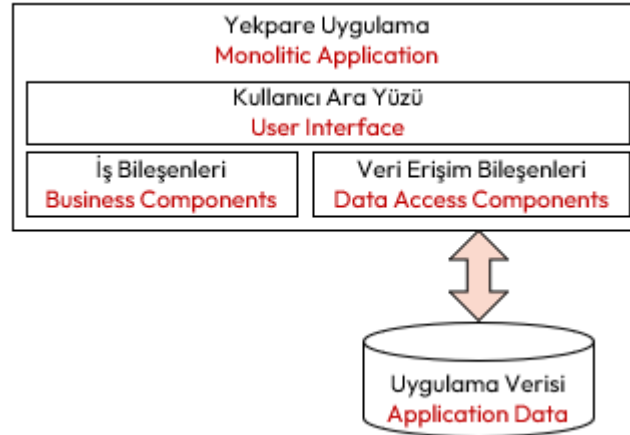
19.4 Katmanlı Mimari

Katman kavramı iki türlü düşünülmelidir. Kodların gruplanarak birbirinden ayrıldığı katmanlara kod katmanı (**layer**) adı verilir. Fiziksel olarak birbirinden ayrılmış kodlara ise fiziksel katman (**tier**) adı verilir. Kısaca derlenmiş olarak birbirinden ayrı olan katmanlar fiziksel katman, diğer tip katman ise kod katmanı olarak adlandırılır. Katman sayısı göz önüne alınarak uygulamalar aşağıdaki şekilde sınıflandırılabilir.

§19.4.1 Yekpare Uygulamalar

Genel olarak hiç katmanı olmayan, tek bir çalıştırılabilir dosyadan oluşan uygulamalar **yekpare (monolithic)** uygulamalar olarak adlandırılır. Aynı bir sınıf kütüphanesi olmayan tüm konsol ve Windows uygulamaları bu sınıfa girer. Şu ana kadar verilen örneklerin çoğu bu kapsamda değerlendirilebilir. Yekpare uygu-

lama, kullanıcı arayüzü, iş mantığı ve veri erişim katmanı gibi tüm bileşenlerin tek bir kod tabanında ve bölünmez bir birim olarak geliştirilip dağıtıldığı geleneksel yazılım mimarisidir. Basit, hızlı geliştirilen ve yönetimi kolay yapılar olsalar da, ölçeklendirme zorluğu ve kod karmaşıklığı gibi dezavantajları bulunur (Şekil 19.1).



Şekil 19.1: Yekpare Uygulama

Yekpare uygulama örneğimizin yapısı aşağıda verilmiştir;

```
KonsolUygulaması/ → Çözüm (Solution)
├─ Yekbare/ → Proje (Console Application Project)
└─ Program.cs → Ana (Main) Program
```

Projeyi oluşturmak için; **Visual Studio** üzerinde **File**→**New**→**Project...** seçilerek **KonsolUygulaması** adlı çözüm içinde **Yekpare** adlı projemiz **Console Application** olarak oluşturulur. Oluşan projede **Program.cs** dosyası otomatik olarak oluşur. Bu dosyayı aşağıdaki şekilde değiştiriyoruz;

```
namespace Yekpare
{
    class İşMantığı
    {
        public void İşİşlevi()
        {
            Console.WriteLine("İş İşlevi Çağrıldı.");
        }
    }
    class Client
    {
        static void Main(string[] args)
        {
            İşMantığı işlev = new İşMantığı();
            işlev.İşİşlevi();
        }
    }
}
```

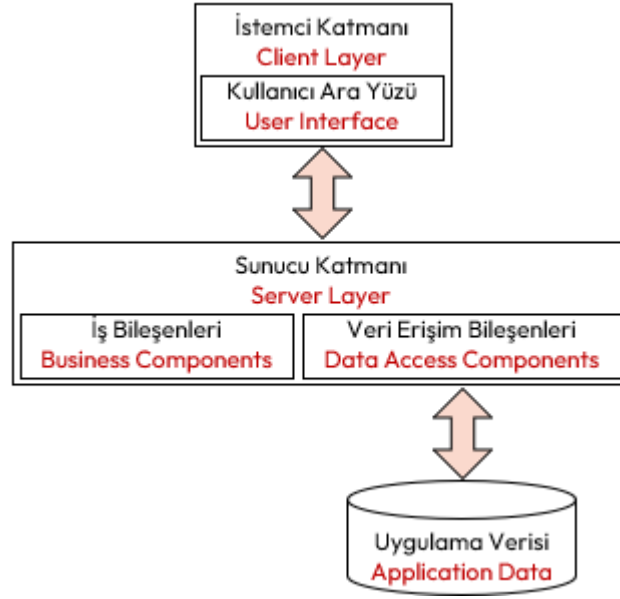
Yekpare uygulamalar, her ne kadar veri tabanı kullansalar da bu tür yazılımlar **tek bir proje olarak kodlanırlar ve derlenir**. Yani **proje tek bir kod tabanından oluşur**. Seksenli yıllara kadar geliştirilen uygulamalar genellikle bu yapıdadır.

Bilgisayara yaptırılan işlerin kapsamının zamanla daha fazla genişlemesi, kurumların iş süreçlerini daha çok içermesi ve uygulamanın işletme ve bakım maliyetlerinin artmasıyla birlikte, bu yazılımları uygun katmanlara bölünmesi ihtiyacı doğurmuştur.

Ş19.4.2 İki Katmanlı Mimari

İki Katmanlı yani **İstemci-Sunucu (client-server)** mimarisinde uygulama iki katmanlı olup, genellikle veri katmanı bir bilgisayarda diğer katman olan istemci katmanı ise kullanıcı bilgisayarlarında bulunur. Bu tür uygulamalarda kullanıcı ara yüzü bileşenleri **UIC (User Interface Components)** istemci (**client**) katmanını oluşturur. Geri kalan iş süreçleriyle ilgili geliştirilen bileşenler ve veri tabanı ile ilgili iletişimi sağlayacak olan bileşenler ise sunucu (**server**) katmanını oluşturur (Şekil 19.2).

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;



Şekil 19.2: İstemci-Sunucu Mimarisi

İstemciSunucuUygulaması/ → Çözüm (Solution)
 | İstemciBileşenleri/ → Proje (Console Application Project)
 | | Program.cs → Ana (Main) Program
 | SunucuBileşenleri/ → Proje (Class Library)
 | | Sunucu.cs → Sınıflar (Class)

Projeyi oluşturmak için;

1. Visual Studio üzerinde **File→New→Project...** seçilerek **İstemciSunucuUygulaması** adlı çözüm içinde **İstemciBileşenleri** adlı projemiz **Console Application** olarak oluşturulur.
2. Oluşan projede Program.cs dosyası otomatik olarak oluşur.

```
namespace İstemciBileşenleri
```

```
{
    class Client
    {
        static void Main(string[] args)
        {
        }
    }
}
```

3. Daha sonra sunucu katmanını yapmak için **Solution Explorer** üzerinde **KonsolUygulaması** çözümüne sağ tıklayarak **Add→New→Project...** seçerek **SunucuBileşenleri** adlı **Class Library** oluşturulur
4. Proje ile oluşan **Class1.cs** dosyasının adı **Sunucu.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace SunucuBileşenleri
{
    public class Server
    {
        public void Sunucuİşlevi()
        {
            Console.WriteLine("Sunucu İşlevi Çağrıldı.");
        }
    }
}
```

5. İstemci tarafında sunucu bileşenlerini kullanmak için **Solution Explorer** üzerinde **SunucuBileşenleri** projesi sağ tıklayarak **Build** seçilir ve derlenir.
6. Ardından **İstemciBileşenleri** projesi seçilir ve **Add→Project Reference→Project...** ile sunucu projemiz istemci projemize dahil edilir.
7. Bunu yaptıktan sonra **İstemciBileşenleri** projesinde aşağıdaki değişiklikler yapılır;

```
using Sunucu; //Bunu yapabilmek için:İstemciye Sunucu Referans olarak eklenir.
namespace İstemciBileşenleri
```



```

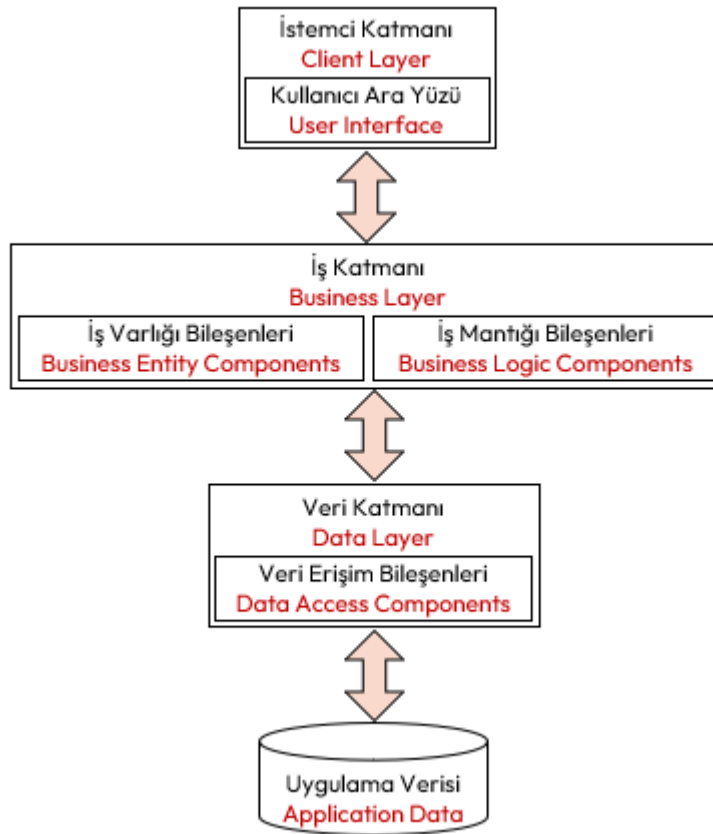
{
    class Client {
        static void Main(string[] args)
        {
            Sunucu.Server sunucu = new Sunucu.Server();
            sunucu.Sunucuİşlevi();
        }
    }
}

```

8. Artık **Solution Explorer** üzerinde **İstemciBileşenleri** projesi sağ tıklanarak **Set as Default Project** seçilir ve program çalıştırılabilir.

§19.4.3 Üç Katmanlı Mimari

Nesne yönelimli yazılım geliştirme ile bildikte nesneler arasındaki iletişimin bir başka katman olarak tasarlandığı uygulamalardır. İki katmanlı mimariden farklı olarak kullanıcı ara yüzü ile iş bileşenleri birbirinden ayrılarak, kullanıcılara istemci yükleme işleminin çok kolaylaştığı uygulamalardır. İstemci-sunucu yazılımlarında zamanla, sunucu tarafında iş süreçlerini modelleyen bileşenler BEC (**Business Entity Components**) ile veri tabanıyla iletişimi sağlayan bileşenler DAC (**Data Access Components**) birbirinden ayrılarak farklı iki katman olarak geliştirilmiştir. Böylece katman sayısı artarak üç katmanlı (**three layer**) mimari hayata geçmiştir (Şekil 19.3).



Şekil 19.3: Üç Katmanlı Mimari

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

```

ÜçKatmanlıUygulama/ → Çözüm (Solution)
├── İstemciKatmanı/ → Proje (Console Application Project)
│   └── Program.cs → Ana (Main) Program
├── İşKatmanı/ → Proje (Class Library)
│   └── İşBileşenleri.cs → Sınıflar (Class)
└── VeriKatmanı/ → Proje (Class Library)
    └── VeriBileşenleri.cs → Sınıflar (Class)

```

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir; Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File**→**New**→**Project...** seçilerek **ÜçKatmanlıUygulama** adlı çözüm içinde **İstemciKatmanı** adlı projemiz **Console Application** olarak oluşturulur.

2. Oluşan projede `Program.cs` dosyası otomatik olarak oluşur.

```
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
        }
    }
}
```

3. Daha sonra sunucu katmanı yapmak için **Solution Explorer** üzerinde **ÜçKatmanlıUygulama** çözümüne sağ tıklanarak **Add→New→Project...** seçerek **İşBileşenleri** adlı **Class Library** oluşturulur
4. Proje ile oluşan `Class1.cs` dosyasının adı **İşBileşenleri.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace İşBileşenleri
{
    public class İşSınıfı
    {
        public void işYöntemi()
        {
            // İş mantığı burada yer alır
        }
    }
}
```

5. Daha sonra sunucu katmanı yapmak için **Solution Explorer** üzerinde **ÜçKatmanlıUygulama** çözümüne sağ tıklanarak **Add→New→Project...** seçerek **VeriKatmanı** adlı **Class Library** oluşturulur
6. Proje ile oluşan `Class1.cs` dosyasının adı **VeriBileşenleri.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace VeriBileşenleri
{
    public class VeriTabanı
    {
        public void veriEkle(int veri)
        {
            // Veri ekleme işlemleri burada yer alır
            Console.WriteLine($"Veri tabanına veri ekleme {veri} işlemi çalıştı.");
        }
        public void veriSil(int veri)
        {
            // Veri silme işlemleri burada yer alır
            Console.WriteLine($"Veri tabanından veri silme {veri} işlemi çalıştı.");
        }
    }
}
```

7. İstemci tarafında sunucu bileşenlerini kullanmak için **Solution Explorer** üzerinde **VeriKatmanı** projesi sağ tıklanarak **Build** seçilir ve derlenir.
8. Ardından **İşKatmanı** projesi seçilir ve **Add→Project Reference→Project...** ile veri katmanı projemiz istemci projemize dahil edilir.
9. Bunu yaptıktan sonra **İşKatmanı** projesinde aşağıdaki değişiklikler yapılır;

```
using VeriBileşenleri; // İş projesine Veri Projesi Referans olarak eklenir.
namespace İşBileşenleri
{
    public class İşSınıfı
    {
        public void işYöntemi()
        {
            // İş mantığı burada yer alır
            int işlenenVeri = 0;
            VeriTabanı veriTabanı = new VeriTabanı(); //
            veriTabanı.veriEkle(işlenenVeri); //
            //...
```

```

        veriTabanı.veriSil(işlenenVeri); //
        Console.WriteLine("İş sınıfı iş yöntemi çalıştı.");
    }

}

```

10. İstemci tarafında iş bileşenlerini kullanmak için **Solution Explorer** üzerinde **İşBileşenleri** projesi sağ tıklanarak **Build** seçilir ve derlenir.
11. Ardından **İstemciKatmanı** projesi seçilir ve **Add**→**Project Reference**→**Project...** ile iş katmanı projemiz istemci projemize dahil edilir.
12. Bunu yaptıktan sonra **İstemciKatmanı** projesinde aşağıdaki değişiklikler yapılır;


```

using İşBileşenleri; // İstemci projesine İş Projesi Referans olarak eklenir.
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
            İşSınıfı işnesnesi = new İşSınıfı(); //
            işnesnesi.işYöntemi(); //
        }
    }
}

```
13. Artık **Solution Explorer** üzerinde **İstemciKatmanı** projesi sağ tıklanarak **Set as Default Project** seçilir ve program çalıştırılabilir.

§19.4.4 Çok Katmanlı Mimari

Yukarıda örneği verilen üç katmanlı uygulamalarda ihtiyaca göre katmanlar daha alt katmanlara ayrılabilir. Örneğin kullanıcı ara yüzü katmanı, aşağıdaki bileşenlerden oluşacak şekilde ikiye ayrılabilir;

1. İnsanların yazılıma talimatlarını verdiği insan talimatları bileşenleri HIC (**Human Instruction Components**) ve
2. Kullanıcı talimatlarını, kullanıcı ara yüzünden alıp, iş katmanının anlayacağı şekle çeviren kullanıcı süreci bileşenleri UPC (**User Process Components**).

Benzer şekilde İş katmanı, aşağıdaki bileşenlerden oluşacak şekilde üçe ayrılabilir;

1. İş akışını sağlayan bileşenler BWC (**Business Workflow Components**),
2. İş varlıkları bileşenleri BEC (**Business Entity Components**) ve
3. İş varlıklarının toplu olarak işlenmesi ile ilgili iş mantığı bileşenleri BLC (**Business Logic Components**)

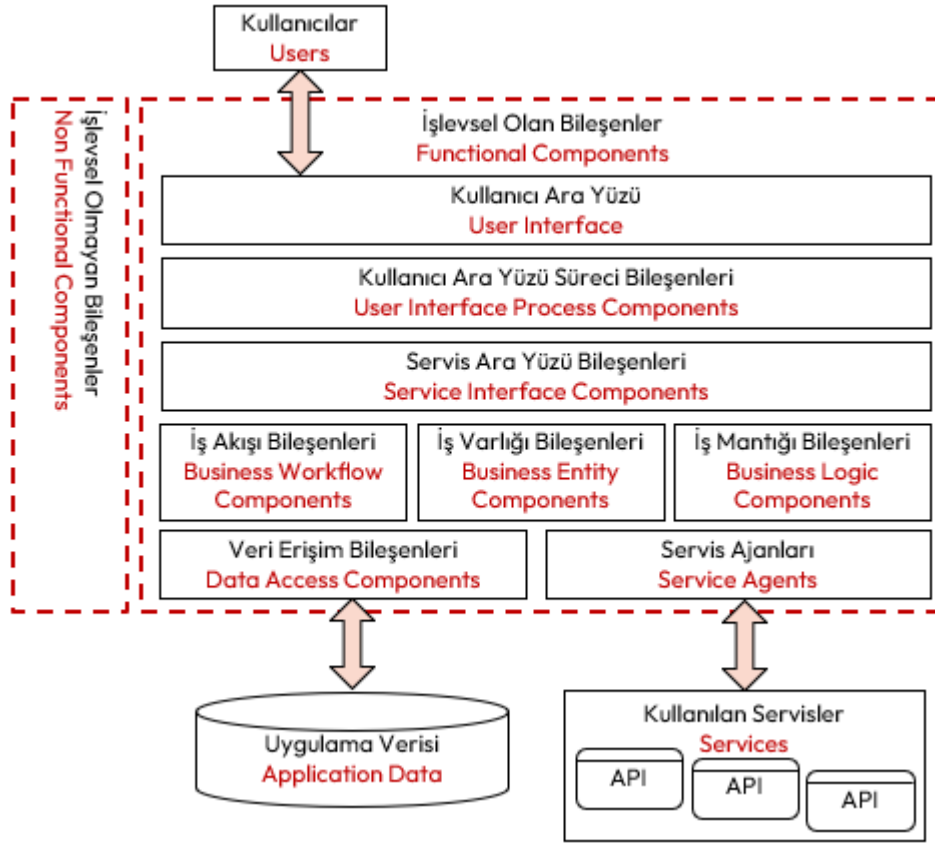
Çok katmanlı mimari, günümüz yazılım mimarisi olup mümkün olduğunca katmanların birbirinden bağımsız tasarlandığı uygulamalardır. Üç katmanlı mimariden daha alt katmanlara ayrılarak geliştirilen yazılımlara, n-katmanlı (**n-layer/n-tier**) yazılım adı verilir. Çok katmanlı yazılım geliştirilirken kullanılan bileşenler ve bu bileşenlerin bulunduğu katmanlar detaylı olarak aşağıdaki şekilde verilmiştir (Şekil 19.4).

Yukarıda verilen şekilde, dikine yazılı olan yerler, yazılımı gerçekleştirilen sistemde bulunmayan, ancak yazılım geliştirilirken ihtiyaç duyulan bileşenleri ve bununla ilgili servisleri temsil etmektedir. Bu nedenle bu bileşenler, yazılımın **niteliksel (işlevsel olmayan) (non-functional)** bölgeleridir. Yani ortada bir yazılım olmayınca hiçbir önemi olmayan bileşenlerdir. Geri kalan bileşenler ise işlevsel (**functional**) bileşenler olarak adlandırılır (Şekil 19.5).

Burada bahsedilen güvenlik (**security**) katmanlara rol bazlı (**role based**) veya kod erişimi (**code access**) olmak üzere iki şekilde ilave edilir.

Sonuç olarak yazılımlarda katman sayısı arttıkça performans (**performance**) düşer ama esneklik (**flexibility**) artar, bakım (**maintainability**) kolaylaşır, yeniden kullanılabilirlik (**reusability**) ve ölçeklenebilirlik (**scalability**) sağlanmış olur.

Çok katmanlı mimari, iş yükünü birden fazla bilgisayar veya sunucu üzerine çeşitli görevlerle dağıtarak yüksek erişilebilirlik sağlar. Bu yaklaşım, sistemin **ölçeklenebilirliğini (scalability)** ve **hata toleransını (fault tolerance)** maksimize eder. Günümüzde bu özellik kendini en net **mikroservis mimarileri** ve **bulut bilişim (cloud computing)** teknolojilerinde gösterir.



Şekil 19.4: Çok Katmanlı Mimari

§19.4.5 Katmanların Tasarlanması

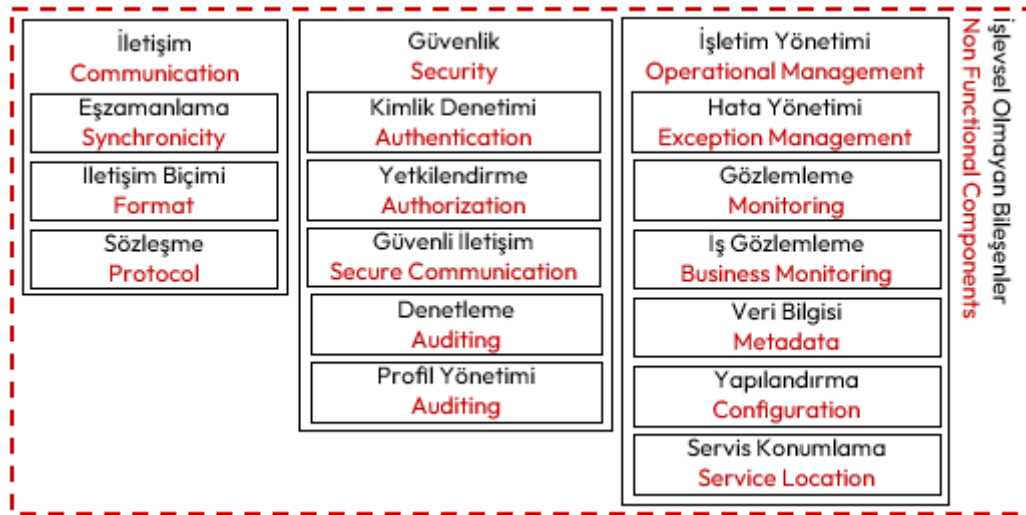
Gereksinim analizinden sonra katmanlar ayrı ayrı tasarlanır;

Business Logic Components (BLC)

Bu katman, sorun alanı bileşenleri PDC (**Problem Domain Components**) olarak da adlandırılır. Uygulamanın, iş modelinde belirtilen, iş kurallarını içeren bileşenlerdir. PDC, gerçek dünyadaki problemin yazılımda modellenmiş haliyle ilgilidir. Sistem çözümünün temel mantığını ve iş kurallarını (**business rules**) kapsar. Gerçek hayat problemini, teknik detaylardan bağımsız şekilde doğru ve tam olarak modellemek amacıyla tanımlanırlar.

```
public class Sipariş //Örnek PDC/BLC Bileşenleri
{
    public Guid Id { get; private set; }
    public List<SiparişKalemi> SiparişKalemleri { get; private set; }
    public DateTime SiparişZamanı { get; private set; }
    public Sipariş()
    {
        Id = Guid.NewGuid();
        SiparişKalemleri = new List<SiparişKalemi>();
        SiparişZamanı = DateTime.UtcNow;
    }
    public void YeniKalemEkle(SiparişKalemi item)
    {
        if (item.Miktar <= 0)
            throw new ArgumentException("Miktar 0 dan fazla olmalıdır!");

        SiparişKalemleri.Add(item);
    }
    public decimal ToplamSiparişÜcretiGetir()
    {
        return SiparişKalemleri.Sum(i => i.BirimFiyat * i.Miktar);
    }
}
```



Şekil 19.5: Niteliksel (İşlevsel Olmayan) Bileşenler

```

}
public class SiparişKalemi
{
    public Guid KalemId { get; set; }
    public int Miktar { get; set; }
    public decimal BirimFiyat { get; set; }
}

```

Business Entity Components (BEC)

Bu katman, veri tabanı tabloları ile eşleşen ve iş nesnelerini temsil eden sınıflar içerir. Sınıflar yalnızca özellik barındırır ve veri taşıma, saklama veya iletme süreçlerinde kullanılır, içinde iş mantığı barındırmaz. Bu sınıflar çok nadir olarak doğrulama (**validation**) yöntemleri içerir. Bu sınıflardan imal edilen nesneler veri transfer nesneleri DTO (**Data Transfer Object**) olarak kullanılır.

```

//Örnek BEC Bileşenleri
public class SiparişDto
{
    public Guid Id { get; set; }
    public List<SiparişKalemiDto> SiparişKalemleri { get; set; }
    public decimal ToplamÜcret { get; set; }
    public DateTime SiparişZamanı { get; set; }
}

public class SiparişKalemiDto
{
    public Guid KalemId { get; set; }
    public int Miktar { get; set; }
    public decimal BirimFiyat { get; set; }
}

```

Problem Domain Components (PDC)

PDC bileşenleri, iş süreçlerini işlerken kullanılabilir; fakat veri tabanına yazılacağı veya servis üzerinden bir istemciye gönderileceği zaman iş varlığına (BEC) dönüştürülür. Dönüştürme için ayrı bileşenler tasarlanır.

```

//PDC den BEC bileşenine dönüştürme
public static class SiparişMapper
{
    public static SiparişDto ToDto(Sipariş sipariş)
    {
        return new SiparişDto
        {
            Id = sipariş.Id,
            SiparişZamanı = sipariş.SiparişZamanı,

```



```

        SiparişKalemleri = sipariş.SiparişKalemleri.Select(i => new SiparişKalemDto
        {
            KalemId = i.KalemId,
            Miktar = i.Miktar,
            BirimFiyat = i.BirimFiyat
        }).ToList(),
        ToplamÜcret = sipariş.ToplamSiparişÜcretiGetir()
    };
}
}

```

Business Workflow Components (BWC)

Bu katman, kurumsal yazılım mimarilerinde iş süreçlerinin (**business processes/workflows**) akışını ve yönetimini tanımlayan bileşenlerdir. Genellikle iş akışlarının adım adım yürütülmesi, durumların yönetilmesi ve süreç bazlı kararların verilmesi gibi konuları kapsar.

```

public class SiparişWorkflow
{
    private readonly İÖdemeServisi ödemeServisi;
    private readonly İStokServisi stokServisi;
    private readonly İGönderimServisi gönderimServisi;

    public OrderWorkflow(İÖdemeServisi ödeme, İStokServisi stok, İGönderimServisi gönderim)
    {
        ödemeServisi = ödeme;
        stokServisi = stok;
        gönderimServisi = gönderim;
    }

    public async Task ProcessOrderAsync(Sipariş sipariş)
    {
        if (!await ödemeServisi.Charge(sipariş))
            throw new Exception("Ödeme Alınamadı!");

        if (!await stokServisi.CheckStock(sipariş.SiparişKalemleri))
            throw new Exception("Yetersiz Stok!");

        await gönderimServisi.DağıtımHazırla(sipariş);

        sipariş.Durum = SiparişDurumu.DağıtımAçıkacak;
    }
}

```

Service Interface Components (SIC)

Servis arayüzü bileşenleri SIC (**Service Interface Components**) katmanı, uygulamanın dış dünya (kullanıcılar, sistemler, API istemcileri vs.) ile iletişim kurduğu giriş noktalarıdır. Diğer bir deyişle, uygulamanın hizmetlerini dışa açan ara yüzlerdir. RESTful API uygulamasının anlatıldığı 19.6.3 başlığını inceleyiniz.

Data Access Components (DAC)

Veri erişim bileşenleri DAC (**Data Access Components**) katmanı, iş varlıklarının (BEC) tuttuğu verilere ilişkin bir kalıcılık sağlamak için veritabanı ya da diğer kalıcı veri kaynaklarına erişimi soyutlayan ve yöneten bileşenlerdir. Kalıcılığı sağlamak için bu bileşenler bazen detaylı bir mantık içerebilir. Bu mantığa, kalıcılık mantığı (**persistence logic**) adı verilir. Bunlar sayesinde uygulamanın diğer katmanları (özellikle uygulama ve iş mantığı katmanları), veri kaynağının teknik detaylarını bilmeden veri okuma/yazma işlemlerini gerçekleştirebilir. Burada veri işlemek için **depo deseni** (**repository pattern**) kullanılır. Bu desende CRUD (**Create-Read-Update-Delete**) işlevlerini de içine alan ara yüzler (**interface**) üzerinden iş bileşenleri (BEC) ihtiyaçları çözülür.

Veri erişim bileşenleri, her BEC ile ilgili veri kaynağında, yeni kayıt (create), mevcut kaydı okuma (read), kaydı güncelleme (update) ve bir kaydı silme (delete) yöntemlerini yerine getirir. Açıklanan bu işlevleri yerine getiren yöntemlerin tümü kısaca CRUD olarak adlandırılır.

CRUD yöntemleri; ABCD (**Add-Browse-Change-Delete**), ACID (**Add-Change-Inquire-Delete**), BREAD (**Browse-Read-Edit-Add-Delete**) ve VADE (**View-Add-Delete-Edit**) olarak da adlandırılır.

```
public interface ISiparişRepository
{
    Task<Sipariş?> GetByIdAsync(Guid id);
    Task AddAsync(Sipariş sipariş);
    Task SaveChangesAsync();
    Task<List<Sipariş>> GetAllAsync();
    Task DeleteAsync(Guid id);
}

public class SiparişRepository : ISiparişRepository
{
    private readonly string _connectionString;
    public SiparişRepository(string connectionString)
    {
        _connectionString = connectionString;
    }
    public async Task<List<Sipariş>> GetAllAsync()
    {
        var sipariş = new List<Sipariş>();
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand("SELECT Id, SiparişZamanı FROM Siparişler", conn);
        await conn.OpenAsync();
        using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            sipariş.Add(new Sipariş
            {
                Id = reader.GetGuid(0),
                SiparişZamanı = reader.GetDateTime(2)
            });
        }
        return sipariş;
    }
    public async Task<Sipariş?> GetByIdAsync(Guid id)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand("SELECT Id, SiparişZamanı FROM Siparişler WHERE Id = @Id", conn);
        cmd.Parameters.AddWithValue("@Id", id);
        await conn.OpenAsync();
        using var reader = await cmd.ExecuteReaderAsync();
        if (await reader.ReadAsync())
        {
            return new Sipariş
            {
                Id = reader.GetGuid(0),
                SiparişZamanı = reader.GetDateTime(2)
            };
        }
        return null;
    }
    public async Task AddAsync(Sipariş order)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand("INSERT INTO Siparişler (Id,SiparişZamanı) VALUES (@Id, @SiparişZamanı)", conn);
        cmd.Parameters.AddWithValue("@Id", order.Id);
        cmd.Parameters.AddWithValue("@SiparişZamanı", order.SiparişZamanı);
    }
}
```

```

        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
    public async Task UpdateAsync(Sipariş order)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand( "UPDATE Siparişler SET SiparişDate = @SiparişZamanı WHERE
        ↳ Id = @Id", conn);
        cmd.Parameters.AddWithValue("@Id", order.Id);
        cmd.Parameters.AddWithValue("@CustomerName", order.CustomerName);
        cmd.Parameters.AddWithValue("@SiparişDate", order.SiparişDate);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
    public async Task DeleteAsync(Guid id)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand( "DELETE FROM Siparişler WHERE Id = @Id", conn);
        cmd.Parameters.AddWithValue("@Id", id);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
}

```

Katmanlar Arası İletişim

Katmanlı mimaride, iş varlığı bileşenleri (BEC) orta katmanda yer aldığından hem kullanıcı katmanında hem de veri erişim katmanında kullanılırlar. Bu nedenle bu katmanlarda iş varlıklarını, verileri, katmanlar arasında taşırlar. Bu taşınma işlemi, genel olarak üç değişik teknoloji ile sağlanır;

1. **Sınıf** : Katmanların birbirine çok yakın olduğu durumda tercih edilir.
2. **Veri Seti (DataSet)**: Katmanlar arası veri iletişimi veri tabanı üzerinden yapılır. Genellikle aynı ağı kullanan katmanlar için tercih edilir.
3. **Serileştirme (serialization)**: Bir nesnenin durumunu (**state**) kalıcı veya taşınabilir bir şekle dönüştürme sürecidir. Katmanların birbirinden uzak olduğu durumda tercih edilir.

Buradaki tercihler uygulamanın hızını etkiler. Dolayısıyla iş varlıkları tasarlanırken yukarıdaki maddelerden doğru olanını tercih etmek gerekir. Serileştirme, 16.8 başlığında işlenmişti.

Katmanlar Arası İstisna İletimi

Çok katmanlı mimaride, katmanların birinde olan istisna, kullanıcı katmanına kadar yeniden fırlatılır (**re-throw**). Aksi takdirde istisna kullanıcı katmanına ulaşmadan program kilitlenir. Kullanıcı istisnanın ne olduğu konusunda bilgi sahibi olamaz. Aşağıda istemci sunucu mimarisinde istisna iletimi örneği verilmiştir.

```

//Sunucu.cs (Class Library)
namespace Sunucu
{
    public class Server {
        public int Bölme(int p1,int p2){
            int ret = 0;
            try {
                ret = p1 / p2;
            } catch (Exception e){
                throw e;
                //hata iletme(rethrow):hata kullanıcı
                //katmanına iletilir.
            }
            return ret;
        }
    }
}

```

Bu sunucu kodunu kullanan istemci uygulaması da aşağıda verilmiştir;

```

//İstemci.cs (Console Application)
using Sunucu; //Bunu yapabilmek için:İstemciye Sunucu Referans olarak eklenir.

```

```
namespace İstemci
{
    class Client {
        static void Main(string[] args){
            Sunucu.Server sunucu = new Sunucu.Server();
            sunucu.Bölme(12,0); //Hata burada oluşur.
        }
    }
}
```

19.5 Bileşen Kullanımı

C# dilinde bileşen kullanımı, yazılımın **modülerlik** (**modularity**) ve **bakım kolaylığı** (**maintainability**) hedeflerine hizmet eder. Bu süreçteki temel araçlar şunlardır:

- ♦ **Arayüz Tabanlı Tasarım** (**interface-based design**): Bileşenlerin birbirine sıkı sıkıya bağlı (**tightly coupled**) olmasını engelleyerek, sistemin esnekliğini artırır.
- ♦ **Paket Yönetimi** (**NuGet**): Hazır bileşenlerin sisteme entegrasyonunu ve sürüm yönetimini standartlaştırır.
- ♦ **Bağımlılık Enjeksiyonu** (**Dependency Injection**): Bileşenlerin yaşam döngülerini merkezi bir yapı üzerinden yöneterek, test edilebilirliği (**testability**) maksimize eder.

Bu yaklaşım, C# geliştiricisine altyapı detaylarıyla uğraşmak yerine, projenin **iş süreçlerine** (**business domain**) odaklanma fırsatı sunar.

§19.5.1 Sınıf Kütüphaneleri ve DLL Yapısı

C#'ta bileşen kullanımının en fiziksel hali **Class Library** (**.dll**) projeleridir. Bir iş mantığını (örneğin; ödeme yönetimi veya iz kaydı) ayrı bir kütüphane olarak derlediğinizde, derlenmiş bu bileşeni projenize sadece referans vererek dahil edebilirsiniz.

Bileşenin iç işleyişi gizlidir; kullanıcı sadece izin verilen **public** metotlarla etkileşime girer (kapsülleme).

§19.5.2 NuGet ve Paket Yönetimi

C# ekosisteminde bileşen kullanımı denilince akla ilk gelen **NuGet** paket yöneticisidir. NuGet, dünya çapındaki geliştiricilerin hazırladığı bileşenlerin "mağazası" gibidir. Kendi yazdığınız bir bileşeni paketleyip şirket içi bir sunucuya veya genel NuGet galerisine yükleyerek binlerce projede saniyeler içinde kullanılabilir hale getirebilirsiniz.

§19.5.3 Arayüzler ile Gevşek Bağlılık

Bileşen tabanlı mimarinin C# tarafındaki kalbi **interface** yapılarıdır. Bir bileşeni kullanırken doğrudan o sınıfa bağımlı olmak yerine bir arayüze bağımlı olursunuz. Örnek: **ILogger** arayüzünü kullanan bir bileşeniniz varsa, yarın öbür gün iz kayıtlarını dosyaya yazan bileşeni söküp, yerine veritabanına yazan bileşeni takabilirsiniz. Kodun geri kalanı bu değişimden etkilenmez.

§19.5.4 Bağımlılık Enjeksiyonu

Modern C#, özellikle .NET Core ve sonrası, bileşen kullanımını **bağımlılık enjeksiyonu** (**dependency injection**) konteyneri ile standart hale getirmiştir. Uygulama çalışma zamanında (**run-time**) hangi bileşenin (servisin) kullanılacağına karar verir. Bu, bileşenlerin birbirine "geçmeli" bir yapıda (Lego parçaları gibi) çalışmasını sağlar.

§19.5.5 Kullanıcı Arayüzü Bileşenleri

C# sadece arka planda değil, kullanıcı arayüzünde UI (**user interface**) de bileşen mantığını kullanır:

- ♦ **Blazor**: Web tarafında tamamen "component-based" bir yapı sunar (örneğin; bir butonu veya tabloyu `<MyCustomTable />` şeklinde bileşen olarak tanımlayıp her yerde kullanabilirsiniz).
- ♦ **WinForms/WPF**: "Custom Control" yapıları ile kendi görsel bileşenlerinizi oluşturmanıza olanak tanır.

19.6 Dağıtık Programlama

Dağıtık Programlama (**distributed programming**), tek bir bilgisayarın kaynaklarının yetmediği veya sistemin kesintisiz çalışmasının kritik olduğu durumlarda, iş yükünün ağ üzerinden birbirine bağlı birden fazla bilgisayara (node) paylaştırılmasıdır. Modern dünyada artık "süper güçlü tek bir bilgisayar" yerine, "birlikte çalışan binlerce ekonomik bilgisayar" felsefesi hakimdir. Dağıtık programlamayı şu üç ana avantajıyla tanımlayabiliriz:

1. Yatay Ölçeklenebilirlik (**horizontal scaling**): Sisteme yeni bir sunucu ekleyerek kapasiteyi artırabilirsiniz.
2. Hata Toleransı (**fault tolerance**): Bir bilgisayar bozulduğunda, diğerleri onun görevini devralır; sistem kapanmaz.
3. Düşük Gecikme (**low latency**): Veriyi kullanıcıya en yakın sunucudan göndererek hızı artırabilirsiniz (**edge computing**).

Mantıksal katmanların birbirinden fiziksel olarak ayrıldığı ve farklı ortamlar üzerinde konumlandırıldığı sistemlere dağıtık uygulamalar (**distributed application**) adı verilir. Dağıtık yazılımlarda iş yükü tek bir kod bloğu veya tek bir merkezi bilgisayar tarafından yürütülmez; bunun yerine her bir katman farklı bir donanım, sunucu veya süreç (**process**) üzerinde koşturulur. İstemci ile sunucu mimarisinin birbirinden ayrıldığı dağıtık yazılımlara olan ihtiyaç, temel olarak şu gerekçelere dayanmaktadır:

- ♦ **Karmaşıklığın Yönetimi**: Bazı büyük ve karmaşık sistemler, işleme güçlerini optimize edebilmek için dağıtık yapılar üzerinde çalışmak zorundadır.
- ♦ **Maliyet ve Performans Avantajı**: Tek bir süper bilgisayar yerine, ağ üzerinden birbirine bağlı çok sayıda ekonomik bilgisayarın işlemci gücünü kullanmak (**parallel processing**), maliyet-performans açısından büyük üstünlük sağlar.
- ♦ **Ölçeklenebilirlik** (**scalability**): Dağıtık mimariler, artan kullanıcı yükü karşısında sisteme yeni düğümler (**node**) eklenerek kapasitenin kolayca artırılmasına imkân tanır.
- ♦ **Donanım Özellikleri**: Bazı özel yazılım bileşenleri, yalnızca belirli donanım yeteneklerine veya işletim sistemlerine sahip sunucularda çalışabilmektedir.
- ♦ **Güvenlik ve İzolasyon**: Kritik verileri barındıran servislerin, güvenlik gerekçesiyle dış dünyaya açık servislerden fiziksel olarak ayrı sunucularda (örneğin; **DMZ** bölgelerinde) tutulması zorunludur.
- ♦ **Hizmet Odaklılık SOA** (**Service Oriented Architecture**): Bazı servislerin sadece birer kaynak veya merkezi API sağlayıcısı olarak konumlandırıldığı, birçok farklı uygulamanın bu merkezi servislerden beslendiği senaryolar.

§19.6.1 Dağıtık Programlama Teknolojileri

Dağıtık uygulamalarda, farklı konumlardaki bileşenlerin birbiriyle veri alışverişi yapabilmesi için tarihsel süreçten günümüze pek çok teknoloji geliştirilmiştir. Bu teknolojiler, iletişimin gerçekleştiği kapsama göre sınıflandırılabilir:

- ♦ **Süreç İçi** (**In-process communication**): Aynı uygulama süreci (**process**) içinde çalışan bileşenler için özel bir iletişim teknolojisine ihtiyaç duyulmaz; doğrudan bellek erişimi ve metod çağrıları kullanılır.
- ♦ **Süreçler Arası İletişim** (**Out-process/IPC-Inter Process Communication**): Aynı işletim sistemi üzerinde fakat farklı süreçlerde çalışan uygulamalar arası iletişimdir. Pano (**clipboard**), Paylaşılan Bellek (**shared memory**), LPC (**Lightweight Procedure Call**) gibi yöntemlerin yanı sıra; COM, DCOM, CORBA ve Java tarafında RMI gibi teknolojiler bu ihtiyacı karşılamak için kullanılmıştır.
- ♦ **Ağlar Arası ve Uzak İletişim** (**Inter-network communication/RPC- remote process communication**): Farklı bilgisayarların bir ağ üzerinden konuşmasını sağlayan yapılardır. Geçmişte .NET Remoting ve JAX-WS gibi protokoller yayginken; WCF (**Windows Communication Foundation**) bu süreçleri standartlaştıran en kapsamlı kütüphanelerden biri olmuştur.

Günümüzde bağlantı hızlarının ve işlem kapasitelerinin artmasıyla birlikte, yukarıdaki teknolojilerin bir kısmı yerini daha hafif, esnek ve yüksek performanslı modern yaklaşımlara bırakmıştır:

- ♦ **RESTful Web Servisleri**: HTTP protokolü üzerinden JSON veya XML formatında veri alışverişi sağlar. ASP.NET Core Web API bu servislerin oluşturulmasında endüstri standardı haline gelmiştir.
- ♦ **gRPC** (**Google Remote Procedure Call**): HTTP/2 protokolü ve **Protocol Buffers** (kısaca **protobuf**) kullanarak çok yüksek performanslı ve ikili (**binary**) formatta iletişim sağlar. Grpc.AspNetCore kütüphanesi ile modern sistemlerde mikroservisler arası hızlı veri aktarımı için tercih edilir.
- ♦ **Mesaj Kuyrukları** (**message queues**): Uygulamalar arasında asenkron (eşzamansız) haberleşmeyi mümkün kılar. RabbitMQ, Apache Kafka, ActiveMQ ve Azure Service Bus sistemin kopukluklara karşı dirençli (**resilience**) olmasını sağlar.
- ♦ **Gerçek Zamanlı İletişim** (**SignalR**): Sunucunun istemciye anlık veri ittiği (push notification), sohbet veya borsa uygulamaları gibi senaryolar için kullanılır.

§19.6.2 Tekdüzen Kaynak Bulucu ve Port

Her bir bilgisayar, ağ üzerinde bir IP (**Internet Protocol**) adresine sahiptir. Bu adresleri ezberlememek için bilgisayarın IP adresi yerine www.ornek.org gibi **alan adı** (domain name) adlı bir metin kullanılır. Bu alan adı ile IP eşleştirmesini **alan adı sunucuları** DNS (**Domain Name Server**) yapar.

İnternete açık bir bilgisayardaki web sayfaları, görseller veya dosyalar gibi her bir kaynağa alan adıyla birlikte kullanılan **tekdüzen kaynak bulucu** URL (**Uniform Resource Locator**) adlı bir metin kullanılır. Kısaca bilgisayarlardaki her bir kaynak da URL olarak gösterilebilir. Aşağıdaki ilk iki satırdaki kaynak aynı kaynaktır

çünkü varsayılan olarak sunucudaki `index.html` dosyası aranır. Bu sunucu ayarı istenirse değiştirilebilir.

```
http://www.ornek.org/
http://www.ornek.org/index.html
http://www.ornek.org/CSS/index.css
http://www.ornek.org/Images/logo.jpg
```

Portlar uygulamaların bilgisayarda çalıştığını gösteren sanal kapılardır. Bilgisayarlar genellikle tek bir bağlantı (**single connection**) ile ağa bağlanırlar. Buradaki tek bağlantıya sahip bilgisayara ağ üzerinden gelen paketlerin, hangi uygulamaya ait olduğunun belirlenmesi ve söz konusu paketin ilgili uygulamaya ulaştırılmasının sağlanması amacıyla portlar kullanılır.

```
http://www.ornek.org:80/
http://www.ornek.org:8080/
ftp://ftp.ornek.com:21/
```

Portlar, 0 ile 65535 arasında olup, ilk 1024 tanesi özel uygulamalar için ayrılmıştır. Örneğin 80 numaralı port **HTTP (Hyper Text Transmission Protocol)** sunucusunu, 25 numaralı port **SMTP (Simple Mail Transport Protocol)** sunucusunu ve 21 numaralı port ise **FTP (File Transfer Protocol)** sunucusunu temsil eder. URL'nin başındaki `http` ve `ftp` ise bilgisayara ulaşılan **port** üzerinde çalışan protokolü göstermektedir.

§19.6.3 Web API ve RESTful Mimari

Web Servis, internet üzerinden farklı uygulamalar arasında platform bağımsız veri alışverişi sağlayan bir yapıdır. C# ekosisteminde web servis oluşturmanın modern ve en verimli yolu **ASP.NET Core Web API** kullanmaktır. Bu platform; yüksek performansı, açık kaynak kodlu oluşu ve platformlar arası (**cross-platform**) çalışma desteği nedeniyle günümüzde endüstri standardı haline gelmiştir.

REST Mimari Stili

REST (**Representational State Transfer**), 2000 yılında *Roy Fielding* tarafından tanımlanmış bir mimari stildir. REST, web sunucuları ile istemciler arasındaki iletişimi sağlayan HTTP protokolünün, özüne sadık kalarak veri transferinde kullanılması esasına dayanır. REST kurallarına uygun olarak tasarlanan API'lere RESTful API denir. Bu mimari her şey bir kaynak (**resource**) olarak kabul edilir. Kaynaklar, URL'deki alan adından sonraki kısma benzeyen, bilgisayarda benzersiz birer URI (**Uniform Resource Identifier**) adını alır.

Yöntem	Görevi	Örnek URI	Açıklama
GET	Veri Oku	/ogrenciler	Tüm öğrencilerin listesini getirir.
GET	Belirli Veriyi Oku	/ogrenciler/5	ID değeri 5 olan öğrenciyi getirir.
POST	Veri Oluştur	/ogrenciler	Yeni bir öğrenci kaydı oluşturur.
PUT	Tam Güncelleme	/ogrenciler/5	Belirtilen kaynağın tüm verilerini günceller.
PATCH	Kısmi Güncelleme	/ogrenciler/5	Kaynağın sadece belirli alanlarını (e-posta gibi) günceller.
DELETE	Veri Sil	/ogrenciler/5	ID değeri 5 olan öğrenci kaydını siler.

Tablo 19.3: HTTP Yöntemlerinin RESTful Servislerde Kullanımı

RESTful Mimari Prensipleri

REST kurallarına uygun olarak tasarlanan API'lere **RESTful API** denir.

- Durumsuzluk (statelessness)**: Her HTTP isteği birbirinden bağımsızdır; sunucu önceki isteklerin bilgisini saklamaz. Oturum veya yetki bilgileri her istekte genellikle JWT (**JSON Web Token**) gibi jetonlar aracılığıyla yeniden iletilir.
- Önbelleklenebilirlik (cacheability)**: Sunucu, yanıtın ne kadar süreyle geçerli olduğunu belirtebilir. Bu, ağ trafiğini azaltır ve performansı artırır.
- Katmanlı Sistem (layered system)**: İstemci, isteğin doğrudan sunucuya mı yoksa bir ara katmana mı (Load Balancer, API Gateway) gittiğini bilmek zorunda değildir. Bu özellik sistemin ölçeklenebilirliğini artırır.
- Veri Formatı**: Veri alışverişinde hafifliği ve hızı nedeniyle JSON tercih edilmekle birlikte, ihtiyaç durumunda XML gibi formatlar da kullanılabilir.

§19.6.4 .NET Core RESTful Web API Örneği

REST, günümüzde istemci (web/mobil) ile sunucu arasındaki iletişimde en yaygın kullanılan standarttır. Okunabilir olması ve HTTP protokolünün tüm nimetlerinden yararlanması en büyük avantajıdır.

.NET Core RESTful Web API örneği olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
WebAPI/ → Çözüm (Solution)
└─ WebApplication1/ → Proje (ASP.NET Core Web API)
```


- Controllers/ → Denetçilerin Olacağı Klasör
 - WeatherForecastController.cs → Class (WeatherForecastController-Controller)
 - KitapKontroller.cs → Class (KitapController-Controller)
- appsettings.json → JSON (Uygulama Ayarları)
- Program.cs → Ana (Main) Program
- WeatherForecast.cs → Class (WeatherForecast-Model)
- WebApplication1.http → HTTP Ayarları

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File→New Project...** seçilerek **WebAPI** adlı çözüm içinde **WebApplication1** adlı projemiz **ASP.NET Core Web API** olarak oluşturulur.
2. Oluşan projede;
Program.cs ve
WeatherForecast.cs ile
Controllers klasöründe bulunan **WeatherForecastController.cs** dosyaları otomatik olarak oluşur.
3. İlk önce **WeatherForecast.cs** dosyası incelendiğinde MVC mimari desenindeki model olduğu görülmektedir.

```
namespace WebApplication1
{
    public class WeatherForecast
    {
        public DateOnly Date { get; set; }

        public int TemperatureC { get; set; }

        public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);

        public string? Summary { get; set; }
    }
}
```

4. Burada sözü geçen **Contollers** dizini altındaki **WeatherController.cs** dosyası MVC mimari desenindeki denetçidir. **WeatherForecastController.cs** dosyası incelendiğinde URL rotası (**route**) üzerinden **GET** isteği ile gelindiğinde 5 adet rastgele hava tahmini üreteceği görülmektedir.

```
using Microsoft.AspNetCore.Mvc;
namespace WebApplication1.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class WeatherForecastController : ControllerBase
    {
        private static readonly string[] Summaries = new[]
        {
            "Freezing", "Bracing", "Chilly", "Cool", "Mild",
            "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
        };

        private readonly ILogger<WeatherForecastController> _logger;

        public WeatherForecastController(ILogger<WeatherForecastController> logger)
        {
            _logger = logger;
        }

        [HttpGet(Name = "GetWeatherForecast")]
        public IEnumerable<WeatherForecast> Get()
        {
            return Enumerable.Range(1, 5).Select(index =>
                new WeatherForecast
                {
                    Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
                    TemperatureC = Random.Shared.Next(-20, 55),
                    Summary = Summaries[Random.Shared.Next(Summaries.Length)]
                }
            );
        }
    }
}
```

- ```

 }).ToArray();
 }
}
}

```
5. Son olarak `Program.cs` dosyasını inceleyelim. `builder.Services.AddControllers();` talimatı ile 18.5 başlığında anlatılan MVC desenindeki `MVCBaşlat()` yöntemi ile yapılan işler burada da web uygulamasının denetçileri `Controllers` klasöründen eklenir. Geri kalan tüm işlemler web sunucusunu `http` isteklerini ayarlamak için ek ayarlardır.
- ```

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();

```
6. Programı çalıştırdığımızda API uygulamamız `http` isteklerini web sunucu üzerinden istekleri kabul etmeye başlar.
- ```

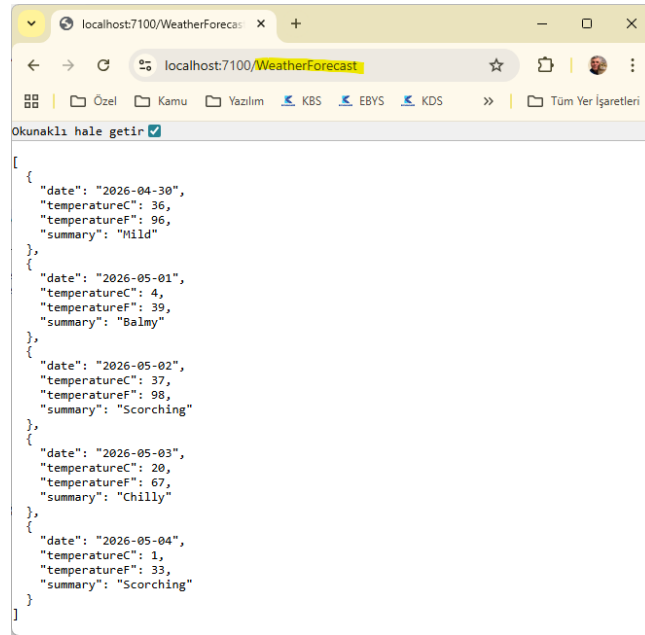
info: Microsoft.Hosting.Lifetime[14]
Now listening on: https://localhost:7100
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5100
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: C:\Users\ilhan\source\repos\WebAPI\WebApplication1

```
7. Çalışan Web API üzerinden veri almak için bir web tarayıcısı açıp <https://localhost:7100/WeatherForecast> adresinden erişebilirsiniz (Şekil 19.6).
8. Şimdi uygulamayı durdurup yeni bir denetçi ekleyelim. Burada model kullanmayacağız; Bunun için **Solution Explorer** üzerinde `Controllers` sağ tıklanarak **Add→Controller...** seçilir ve `KitapController.cs` adında **MVC Controller-Empty** denetçi oluşturulur.
- ```

using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    public class KitapController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}

```



Şekil 19.6: Web API'ye Tarayıcı Üzerinden Erişim

9. Burada model ve gösterim ihtiyacımız olmadığından `Controllers/KitapController.cs` içinde aşağıdaki değişiklikleri yapalım.

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class KitapController : Controller
    {
        [HttpGet]
        public IActionResult TumKitaplariGetir()
        {
            var kitaplar = new[]
            {
                new { Id = 1, Ad = "Sefiller", Yazar = "Victor Hugo" },
                new { Id = 2, Ad = "Suç ve Ceza", Yazar = "Dostoyevski" }
            };

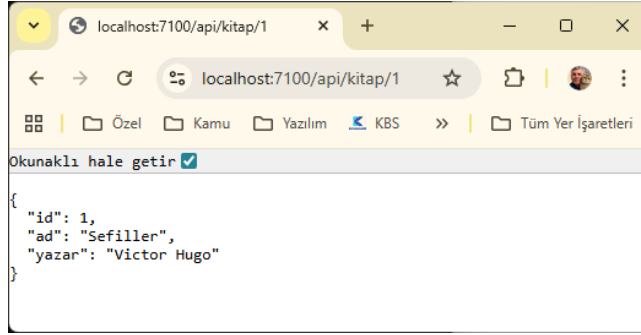
            return Ok(kitaplar);
        }

        [HttpGet("{id}")]
        public IActionResult KitapGetir(int id)
        {
            if (id == 1)
                return Ok(new { Id = 1, Ad = "Sefiller", Yazar = "Victor Hugo" });
            else
                return NotFound("Kitap bulunamadı");
        }
    }
}
```

10. Uygulamayı tekrar çalıştırıp tarayıcıda tüm kitapları getirmek için <https://localhost:7100/api/kitap>, 1 numaralı kitabı getirmek için <https://localhost:7100/api/kitap/1> adresi yazılır (Şekil 19.7).

API'mizden veri almak için örnekteki gibi tarayıcı kullanabiliriz ama bu her zaman yeterli olmaz.

◇ API'mizi test etmek için en yaygın HTTP test aracı olan **Postman**,



Şekil 19.7: Web API ile Kitap Controller'a Tarayıcı Üzerinden Erişim

- ◇ Komut satırından istek gönderme için **cURL**
- ◇ Otomatik API dokümantasyonu da çıkaran **Swagger UI** kullanılabilir.

§19.6.5 Yüksek Performanslı gRPC Uygulama Örneği

gRPC, özellikle mikroservislerin kendi aralarındaki (**internal**) haberleşmesinde tercih edilir. REST'in aksine metin (JSON) değil, ikili formatta veri taşır.

gRPC'de her şey bir **.proto** dosyası (sözleşme) ile başlar. Bu dosyada, servislerin hangi yöntemlere sahip olacağını tanımlanır.

```
syntax = "proto3";
```

```
option csharp_namespace = "FinansServisi.gRPC";
```

```
package finans;
```

```
// Servis tanımı
```

```
service FiyatTakip {
```

```
    // Unary: Tek istek, tek yanıt
```

```
    rpc HisseSenediSorgula(SorguIstegi) returns (FiyatYaniti);
```

```
    // Server Streaming: Bir istek, sürekli veri akışı
```

```
    rpc CanliFiyatTakibi(SorguIstegi) returns (stream FiyatYaniti);
```

```
}
```

```
message SorguIstegi {
```

```
    string sembol = 1;
```

```
}
```

```
message FiyatYaniti {
```

```
    string sembol = 1;
```

```
    double fiyat = 2;
```

```
    string borsa = 3;
```

```
    int64 zaman_damgasi = 4;
```

```
}
```

C# tarafında **gRPC** servisleri, otomatik üretilen sınıfları (**base class**) miras alarak yazılır.

```
using Grpc.Core;
```

```
using FinansServisi.gRPC;
```

```
public class FiyatTakipServisi : FiyatTakip.FiyatTakipBase
```

```
{
```

```
    // Unary Örneği
```

```
    public override Task<FiyatYaniti> HisseSenediSorgula(SorguIstegi request, ServerCallContext  
        ↪ context)
```

```
    {
```

```
        return Task.FromResult(new FiyatYaniti
```

```

    {
        Sembol = request.Sembol,
        Fiyat = 150.25,
        Borsa = "BIST",
        ZamanDamgasi = DateTimeOffset.UtcNow.ToUnixTimeSeconds()
    });
}

// Streaming Örneği: Yüksek performanslı veri akışı
public override async Task CanliFiyatTakibi(SorguIstegi request,
    IServerStreamWriter<FiyatYaniti> responseStream, ServerCallContext context)
{
    var rnd = new Random();
    while (!context.CancellationToken.IsCancellationRequested)
    {
        var yaniti = new FiyatYaniti
        {
            Sembol = request.Sembol,
            Fiyat = 150 + rnd.NextDouble() * 10,
            ZamanDamgasi = DateTimeOffset.UtcNow.ToUnixTimeSeconds()
        };

        await responseStream.WriteAsync(yaniti);
        await Task.Delay(1000); // 1 saniyede bir fiyat güncelle
    }
}
}

İstemci uygulamamızda, gRPC kanalı (channel) üzerinden sunucuya bağlanır.
using Grpc.Net.Client;
using FinansServisi.gRPC;

using var channel = GrpcChannel.ForAddress("https://localhost:5001");
var client = new FiyatTakip.FiyatTakipClient(channel);

// 1. Unary Çağrı
var cevap = await client.HisseSenediSorgulaAsync(new SorguIstegi { Sembol = "THYAO" });
Console.WriteLine($"Tekil Sorgu: {cevap.Sembol} -> {cevap.Fiyat}");

// 2. Streaming Çağrı
using var akis = client.CanliFiyatTakibi(new SorguIstegi { Sembol = "ASELS" });
await foreach (var fiyat in akis.ResponseStream.ReadAllAsync())
{
    Console.WriteLine($"Canlı Güncelleme: {fiyat.Sembol} -> {fiyat.Fiyat}");
}

```

§19.6.6 RESTful Web API ve gRPC Seçim Kriterleri

Özellik	RESTful API	gRPC
Protokol	HTTP 1.1 / HTTP 2	HTTP 2 (Zorunlu)
Veri Formatı	JSON (metin Tabanlı)	Protocol Buffers (ikili)
Performans	Orta (serileştirme yükü var)	Çok Yüksek (düşük CPU yükü)
Okunabilirlik	İnsan tarafından okunabilir	Makine dostu (okunamaz)
Kullanım Alanı	İstemci-Sunucu (Public API)	Mikroservisler arası (internal)

Tablo 19.4: Karşılaştırma ve Seçim Kriterleri

19.7 Modern Dağıtık Programlama: Mikroservis Mimarisi

Mikroservis mimarisi (microservices), yekpare (monolithic) yapıların aksine, bir uygulamayı birbirleriyle hafif protokoller üzerinden haberleşen, bağımsız ve küçük servisler bütünü olarak inşa etme yaklaşımıdır. Her bir servis, tek sorumluluk prensibi (single responsibility) doğrultusunda tasarlanır. Bu mimarinin temel çıktıları; hata izolasyonu (fault isolation), teknoloji bağımsızlığı ve yatay ölçeklenebilirlik (horizontal scala-

bility) olarak özetlenebilir.

§19.7.1 Mimari Katmanlar ve Bileşenler

Bir mikroservis uygulaması, monolitik yapıların aksine aşağıdaki temel bileşenlerin bir araya gelmesiyle oluşur:

1. **API Gateway:** İstemciler ile iç servisler arasında bir **proxy** görevi görür. Güvenlik ve yönlendirme burada merkezileştirilir. Uygulamanın tüm dış isteklerin (mobil uygulama, web tarayıcısı vb.) karşılandığı merkezi noktadır. İstemciler tek bir adresle konuşur, **API Gateway** bu isteği arkadaki ilgili mikroservise yönlendirir. **Kimlik doğrulama** (**authentication**), yük dengeleme (**load balancing**), isteklerin yönlendirilmesi ve hız sınırlama (**rate limiting**)
2. **Microservices:** Kendi veritabanına sahip, bağımsız olarak dağıtılabılır (**deployable**) iş birimleridir. Her biri tek bir işi (**single responsibility**) yapan bağımsız birimlerdir. Örnek: Sipariş Servisi (**Order Service**) sadece siparişleri yönetir, Stok Servisi (**stock service**) sadece envanteri tutar. Her servisin kendi veritabanı (**database per service**) vardır. Bu sayede bir servisin veritabanındaki değişiklik diğerlerini etkilemez. Veri yönetiminde ise her servisin kendi veritabanına sahip olması esastır. Bu durum veri bağımsızlığını sağlasa da, dağıtık sistemlerde **veri tutarlılığı** (**data consistency**) yönetimini zorunlu kılar.
3. **Message Broker:** Servisler arası **gevşek bağlılık** (**loose coupling**) sağlamak için asenkron iletişimi yönetir.
4. **Service Registry:** Dinamik servis adreslerinin tutulduğu ve sorgulandığı merkezi rehberdir (**service discovery**).

Bu yapı, özellikle karmaşık kurumsal sistemlerde **bakım kolaylığı** ve **hızlı yayına alma** (**time to market**) avantajları sağlar.

Burada bahsedilen kavramlar mikroservis mimarisi ile kurumsal yazılım geliştirme yapacakların bilmesi gereken kavramlardır. Zaman kritik olan ve belli bir süre içinde tamamlanması gereken süreçleri içeren yazılımlar için bu mimari tercih edilmez. **Bu kavramlar bir kitap dolduracak kadar geniş anlatılabilir.** Buradaki amacım 19.7 başlığı altında anlatılan kavramlar hakkında farkındalık yaratmaktır.

§19.7.2 Temel Prensipler ve Avantajlar

- ◊ **Bağımsız Ölçeklenebilirlik** (**scalability**): Sistemin tamamı yerine, sadece yoğunluk yaşayan spesifik servisler (örneğin; ödeme veya stok servisi) **yatayda ölçeklenebilir** (**horizontal scaling**).
- ◊ **Teknoloji Çeşitliliği** (**polyglot programming**): Her servis, iş ihtiyacına göre en uygun programlama dili (C++, Python, Go, Java, C#, ...) ve veritabanı yapısı ile geliştirilebilir.
- ◊ **Hata İzolasyonu** (**fault isolation**): Dağıtık yapı sayesinde, bir serviste meydana gelen kritik bir hata tüm sistemin çökmesine neden olmaz; sistemin geri kalanı çalışmaya devam eder (**resilience**).

§19.7.3 Mikroservislerde Haberleşme

Mikroservisler arası iletişim iki temel mekanizma ile yönetilir:

1. **Senkron İletişim:** Genellikle **REST** veya **gRPC** protokolleri üzerinden, bir servisin diğerinden yanıt beklediği durumlarda kullanılır.
2. **Asenkron İletişim:** **RabbitMQ**, **Apache Kafka** veya **Redis** gibi **mesaj kuyrukları** (**message brokers**) kullanılarak servislerin birbirine sıkı sıkıya bağlanması (**tight coupling**) engellenir.

§19.7.4 Mikroservislerde Veri Yönetimi ve Tasarım Desenleri

Dağıtık bir mimaride veritabanlarının ayrıştırılması (**decentralized data management**), beraberinde veri tutarlılığı zorluklarını getirir. Bu zorlukları aşmak için modern yazılım mühendisliğinde şu yaklaşımlar kullanılır:

Saga Tasarım Deseni

Saga deseni (**saga pattern**), dağıtık işlemleri (**distributed transactions**) yönetmek için kullanılır. Bir hata durumunda, sistemin tutarlı bir hale geri dönmesi için her başarılı işlemin bir **telafi edici işlemi** (**compensating transaction**) bulunur.

CQRS ve Event Sourcing

- ◊ **CQRS** (**command query responsibility segregation**): Yazma ve okuma modellerinin ayrılması prensibidir. Bu sayede okuma işlemleri, yazma performansını etkilemeden ölçeklenebilir.
- ◊ **Event Sourcing** (**event sourcing**): Sistemin durumunu bir olaylar dizisi olarak saklama yöntemidir. Bu, geçmişe dönük denetim (**auditing**) ve hata analizi için eşsiz bir veri kaynağı sağlar.

Hizmet Yönetimi ve Orkestrasyon

Mikroservislerin canlı ortamdaki karmaşıklığını yönetmek için **Docker** ve **Kubernetes** gibi araçlar standart haline gelmiştir. Bu araçlar; servislerin otomatik ölçeklenmesi, yük dengeleme ve **hata toleransı (fault tolerance)** süreçlerini otomatikleştirir:

- ◊ Konteyner Yapılandırması (**Docker**): Uygulamayı tüm bağımlılıklarıyla birlikte paketler.
- ◊ Orkestrasyon (**Kubernetes**): Yüzlerce konteynerin otomatik olarak ölçeklenmesini, hata durumunda yeniden başlatılmasını ve yönetilmesini sağlar.

§19.7.5 Operasyonel Zorluklar

Dağıtık mimarinin getirdiği esneklik, beraberinde bazı karmaşıklıkları da getirir:

- ◊ **Hizmet Keşfi (service discovery)**: Dinamik olarak değişen servis adreslerinin otomatik olarak tespiti. Dağıtık bir sistemde servislerin IP adresleri dinamik olarak değişebilir. Hizmet keşfi, hangi servisin nerede çalıştığını bilen bir "telefon rehberi" gibidir.
- ◊ **Dağıtık İzlenebilirlik (distributed tracing)**: Bir isteğin onlarca servis arasındaki yolculuğunun takip edilmesi ve hata ayıklama süreçleri.
- ◊ **Ağ Gecikmesi (network latency)**: Servisler arası iletişimin ağ üzerinden yapılması nedeniyle oluşan performans kısıtları.

19.8 Örnek Kurumsal Uygulama

Kurumlar, içinde oldukları zamana bağlı olarak, iş yapma şekillerini ya da süreçlerini değiştirirler. Buna kullandıkları girdilerdeki değişim, istenilen çıktılardaki değişim ya da yasal mevzuattaki değişim sebep olmaktadır. Buna paralel olarak organizasyonu modelleyen ya da benzetim yapan yazılımların bu değişiklikleri karşılayabilir olması gerekir.

Yazılımlar, değişim olmadığı takdirde, sonsuza kadar çalışabilirler. Ancak yazılımda yapılacak her değişimin, canlı bir varlığa atılan bir bıçak darbesi olacağı akıldan çıkarılmamalıdır. Bu nedenle yazılımlar geliştirilirken, değişim yönetiminin ya da hangi sürecin ne kadar değişebileceğinin önceden belirlenmesi gerekir. Değişim yönetimi ne kadar başarılı olur ise yazılım da o kadar başarılı olacaktır.

Değişim yönetiminin yanında, yazılımda daha önceden yapılmış ve başarı sağlamış olan modellerin kullanılması da oldukça önemlidir. Yani yazılım tasarımı ve analizi yapılırken başarılı tecrübeleri içeren **desenlerin (pattern)** kullanılması gerekir. Bir yazılım, ne kadar fazla desenlere uygun yazılır ise, o kadar değişime açık olacaktır. Bütün bunların yanında yazılımların katmanlara ayrılması işletim ve bakım açısından çok önemlidir. Çok katmanlı mimaride yazılan yazılımlar, ölçeklenebilirliği arttırdığı gibi birden fazla donanıma yükü paylaştırdığı için, küçük fiyatlı donanımlarla büyük işler başarılabılır.

Örnek uygulama olarak müşteriden sipariş alan basit bir uygulama ele alınacaktır. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
KurumsalUygulama/ → Çözüm (Solution)
├── Entities/ → Proje (Class Library)
│   ├── Sipariş.cs → Sınıf (Class)
│   └── DataAccess/ → Proje (Class Library)
│       ├── SiparişRepository.cs → Sınıf (Class)
│       └── Business/ → Proje (Class Library)
│           ├── SiparişYöneticisi.cs → Sınıf (Class)
│           └── UI/ → Proje (Console Application))
│               └── Program.cs → Ana (Main) Program
```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File**→**New**→**Project...** seçilerek **KurumsalUygulama** adlı çözüm içinde **Entities** adlı ilk projemiz **Class Library** olarak oluşturulur.
2. Proje ile oluşan **Class1.cs** dosyasını **Sipariş.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştiriyoruz;

```
//İş Varlığı Bileşenleri (BEC)
//Katmanlar arası taşınacak veri yapısını temsil eder
namespace Entities
{
    public class Sipariş
    {
        public Guid Id { get; set; }
        public string MüşteriAdı { get; set; } = string.Empty;
        public DateTime SiparişZamanı { get; set; }
    }
}
```

- ```

 }
}

```
3. Daha sonra **Solution Explorer** üzerinde çözüme sağ tıklanarak **Add→New Project...** seçilerek **DataAccess** adlı ikinci projemiz **Class Library** olarak oluşturulur.
  4. **Solution Explorer** üzerinde **Entities** projesi sağ tıklanarak **Build** seçilir ve derlenir.
  5. **Solution Explorer** üzerinde bu proje sağ tıklanarak **Add→Project Reference** ile **Entities** projesi dahil edilir.
  6. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages ...** tıklanır ve **Microsoft.Data.Sqlite** paketi yüklenir.
  7. **Visual Studio** üzerinde **Tools→SQLite/SQL Server Compact Toolbox** tıklanır ve **DataAccess** klasöründe SQLite veritabanı **Data.db** oluşturulur. Aynı yerden aşağıdaki sorgu çalıştırılarak **Siparişler** tablosunu oluşturmamız gerekir;
 

```

CREATE TABLE Siparişler (
 Id TEXT PRIMARY KEY, -- GUID string olarak
 MüşteriAdı TEXT NOT NULL,
 SiparişZamanı TEXT NOT NULL -- ISO 8601 tarih formatında saklanabilir
);

```
  8. Daha sonra bu tabloyu veri eklemek ve verileri okumak için kullanabiliriz. Proje ile oluşan **Class1.cs** dosyasını **SiparişRepository.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştirilir;

```

//Veri Erişim Bileşenleri (DAC)
//Veri kaynağında CRUD işlemlerini yapar
using Entities;
using Microsoft.Data.Sqlite;

namespace DataAccess
{
 public class SiparişRepository
 {
 private readonly string connectionString;
 public SiparişRepository(string connectionString)
 {
 this.connectionString = connectionString;
 }
 public List<Sipariş> GetAll()
 {
 var orders = new List<Sipariş>();
 using var conn = new SqliteConnection(connectionString);
 conn.Open();
 using var cmd = conn.CreateCommand();
 cmd.CommandText="SELECT Id, MüşteriAdı, SiparişZamanı FROM Siparişler";

 using var reader = cmd.ExecuteReader();
 while (reader.Read())
 {
 orders.Add(new Sipariş
 {
 Id = reader.GetGuid(0),
 MüşteriAdı = reader.GetString(1),
 SiparişZamanı = reader.GetDateTime(2)
 });
 }
 conn.Close();
 return orders;
 }
 public void Add(Sipariş sipariş)
 {
 using var conn = new SqliteConnection(connectionString);
 conn.Open();
 using var cmd = conn.CreateCommand();
 cmd.CommandText = "INSERT INTO Siparişler (Id, MüşteriAdı, SiparişZamanı)"+
 "VALUES(@Id, @Ad, @Zaman)";
 cmd.Parameters.AddWithValue("@Id", sipariş.Id);

```

```

 cmd.Parameters.AddWithValue("@Ad", sipariş.MüşteriAdı);
 cmd.Parameters.AddWithValue("@Zaman", sipariş.SiparişZamanı);
 cmd.ExecuteNonQuery();
 conn.Close();
 }
}
}

```

9. Ardından **Solution Explorer** üzerinde çözüme sağ tıklanarak **Add→New Project...** seçilerek **Business** adlı üçüncü projemizi **Class Library** olarak oluşturuyoruz.
10. Bu projeye de sağ tıklanarak **Add→Project Reference** ile **Entities** projesi dahil edilir.
11. **Solution Explorer** üzerinde **DataAccess** projesi sağ tıklanarak **Build** seçilir ve derlenir.
12. **Business** projesi sağ tıklanarak **Add→Project Reference** ile **DataAccess** projesi dahil edilir.
13. Proje ile oluşan **Class1.cs** dosyasını **SiparişYöneticisi.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştirilir;

```

//İş Mantığı Bileşenleri (BLC)
//İş kurallarını uygular, doğrulama (validaton) yapar
using Entities;
using DataAccess;
namespace Business
{
 public class SiparişYöneticisi
 {
 private readonly SiparişRepository depo;
 public SiparişYöneticisi(SiparişRepository repo)
 {
 depo = repo;
 }
 public List<Sipariş> GetSiparişler()
 {
 return depo.GetAll();
 }
 public void SiparişOluştur(string müşteriAdı)
 {
 if (string.IsNullOrEmpty(müşteriAdı))
 throw new ArgumentException("Müşteri adı gereklidir");
 var yeniSipariş = new Sipariş
 {
 Id = Guid.NewGuid(),
 MüşteriAdı = müşteriAdı,
 SiparişZamanı = DateTime.UtcNow
 };
 depo.Add(yeniSipariş);
 }
 }
}

```

14. Son olarak **Solution Explorer** üzerinde çözüme sağ tıklanarak **Add→New Project...** seçilerek **UI** adlı dördüncü projemizi **Console Application** olarak oluşturuyoruz.
15. Bu proje sağ tıklanarak **Add→Project Reference...** ile **DataAccess** projesi dahil edilir.
16. **Solution Explorer** üzerinde **Business** projesi sağ tıklanarak **Build** seçilir ve derlenir.
17. **UI** projesine sağ tıklanarak **Add→Project Reference...** ile **Business** projesi dahil edilir.
18. Proje ile oluşan **Program.cs** aşağıdaki şekilde değiştirilir;

```

//Kullanıcı Arayıcı (UI-Console Application)
//Kullanıcı ile etkileşim sağlar

using Business;
using DataAccess;

//Çalıştırılabilir .NET uygulamasının olduğu klasör bulunur. UI Altında.
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
//DataAccess Proje klasörü bulunur.

```

```

var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü,
 ↪ @"..\..\..\DataAccess"));
var db = Path.Combine(projeKlasörü, "Data.Db");
//Bağlantı metni oluşturulur.
var connectionString = $"Data Source={db}";

var depo = new SiparişRepository(connectionString);
var yönetici = new SiparişYöneticisi(depo);

// Yeni sipariş ekleme
Console.Write("Müşteri adı girin: ");
string ad = Console.ReadLine() ?? "";
yönetici.SiparişOluştur(ad);
Console.WriteLine("Sipariş eklendi!");

// Siparişleri listeleme
var siparişler = yönetici.GetSiparişler();
foreach (var sipariş in siparişler)
{
 Console.WriteLine($"{sipariş.Id} - {sipariş.MüşteriAdı} - {sipariş.SiparişZamanı}");
}

```

19. Uygulamayı çalıştırmak için **Solution Explorer** üzerinde **UI** projesi sağ tıklanarak **Set As Startup Project** seçilir. Artık programımız çalışır.

## 19.9 Düşün ve Çöz

- ◊ **Düşün:** İş analizi (**business analysis**) ile gereksinim analizi (**requirements analysis**) arasındaki farkı 'ne istiyoruz?' ve 'bunu nasıl yapacağız?' soruları üzerinden açıklayınız. Bu iki disiplin bir yazılım projesinde nasıl iş birliği yapar?
- ◊ **Düşün:** İki katmanlı (**2-tier**), üç katmanlı (**3-tier**) ve çok katmanlı (**n-tier**) mimari arasındaki farklar nelerdir? Her katmanın değiştirilmesinin diğer katmanları etkilememe ilkesi (**separation of concerns**) pratik olarak nasıl sağlanır?
- ◊ **Düşün:** RESTful API tasarımında kaynak odaklı URL yapısı neden önemlidir? HTTP yöntemlerinin (**GET**, **POST**, **PUT**, **DELETE**) doğru kullanımı, API'nin sezgisel kullanılabilirliğini nasıl artırır?
- ◊ **Düşün:** Mikroservis mimarisi yekpare mimariye kıyasla ne gibi avantajlar sunar? Ancak neden her proje için mikroservis önerilmez? Bir startup'ın neden yekpare başlayıp daha sonra mikroservise geçmesi mantıklı olabilir?
- ◊ **Düşün:** Depo deseni (**repository pattern**) veri erişim kodunu iş mantığından nasıl ayırır? Bu ayrımın birim testi (**unit test**) yazılabilirliği üzerindeki etkisi nedir?
- ◊ **Düşün:** API versiyonlamanın önemi nedir? Yayımlanmış bir API'nin geriye dönük uyumluluğunu (**backward compatibility**) bozmadan nasıl geliştirilir? URI tabanlı (**/v1/**, **/v2/**) ve başlık tabanlı versiyonlama arasındaki fark nedir?
- ◊ **Düşün:** Ön bellek (**cache**) katmanı bir kurumsal uygulamada hangi sorunları çözer? Veri tutarsızlığı (**cache invalidation**) problemini önlemek için hangi stratejiler kullanılabilir?
- ◊ **Düşün:** **MoSCoW** önceliklendirme tekniği gereksinim yönetiminde nasıl kullanılır? 'Olmazsa olmaz' ile 'olsa iyi olur' arasındaki sınırı çizmek neden zordur ve bu zorluk proje gecikmelerine nasıl yol açar?
- ◊ **Çöz:** Bir kütüphane yönetim sistemi için katmanlı mimari tasarlayın: Sunum Katmanı (**konsol**), İş Katmanı (doğrulama ve iş kuralları) ve Veri Erişim Katmanı (dosya veya SQLite) olsun. Kitap ödünç alma ve iade işlemlerini katmanlar arasında gezin.
- ◊ **Çöz:** Basit bir RESTful API istemcisi yazın: **JSONPlaceholder** gibi açık bir API üzerinden **GET**, **POST** ve **PUT** isteklerini **HttpClient** ile gerçekleştirin; yanıtları JSON'dan nesneye dönüştürün.
- ◊ **Çöz:** Repository deseni kullanarak bir ürün yönetim uygulaması tasarlayın: **ÜrünDepo** arayüzü, bellek içi (**in-memory**) ve dosya tabanlı iki gerçekleştirim yazın; iş katmanı hangi depoyu kullandığını bilmesin.
- ◊ **Çöz:** Bir siparişi işleme koyan, stok azaltan ve müşteriyi bilgilendiren üç bağımsız 'servis' tasarlayın. Bu servisleri yekpare (doğrudan çağrı) ve olaya dayalı (**event-driven**, gözlemci deseni) yaklaşımla uygulayıp karşılaştırın.
- ◊ **Çöz:** Basit bir API versiyonlama sistemi tasarlayın: aynı iş mantığına sahip iki sürümü (**/v1/**, **/v2/**) destekleyen bir yönlendirici (**router**) sınıfı oluşturun; v2'de ek alanlar dönsün.